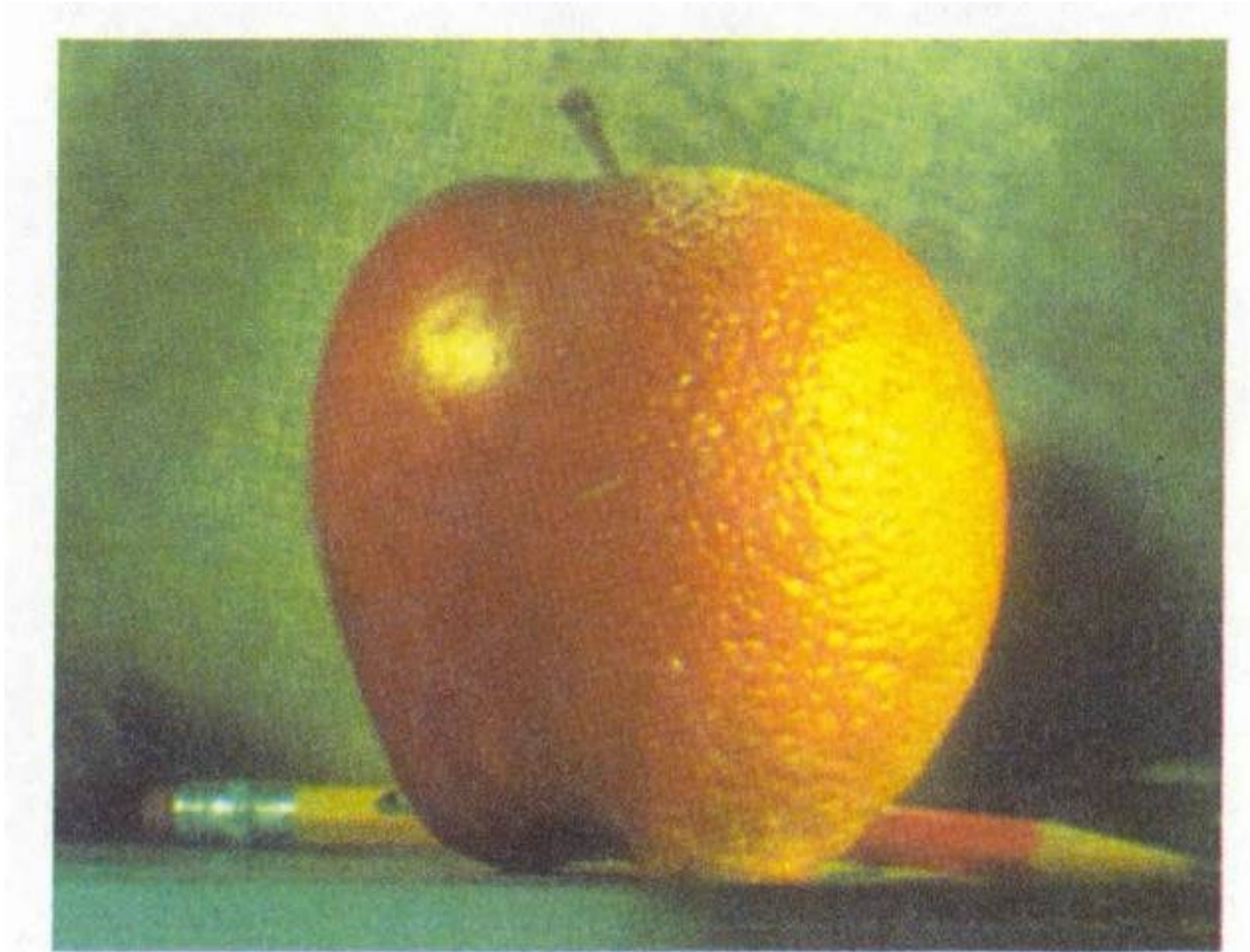


Pyramid Blending, Templates, NL Filters



CS180: Intro to Comp. Vision and Comp. Photo
Alexei Efros and Ren Ng, UC Berkeley, Fall 2026

Think, Pair, Share



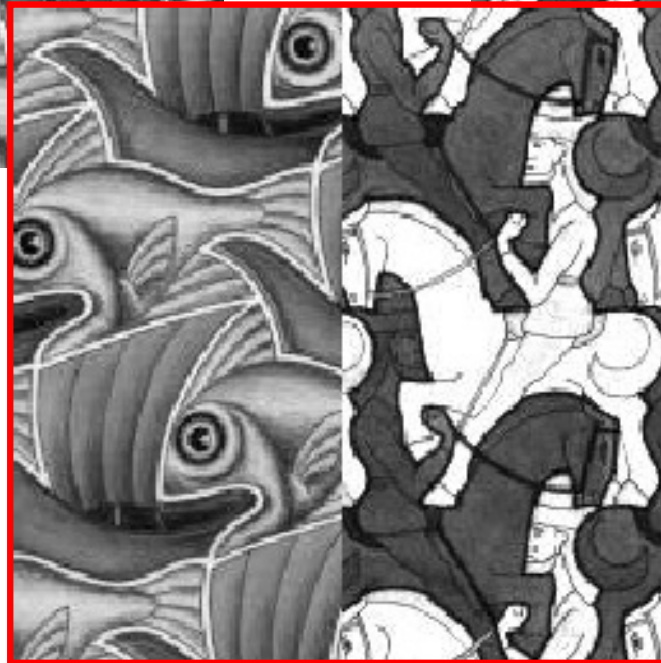
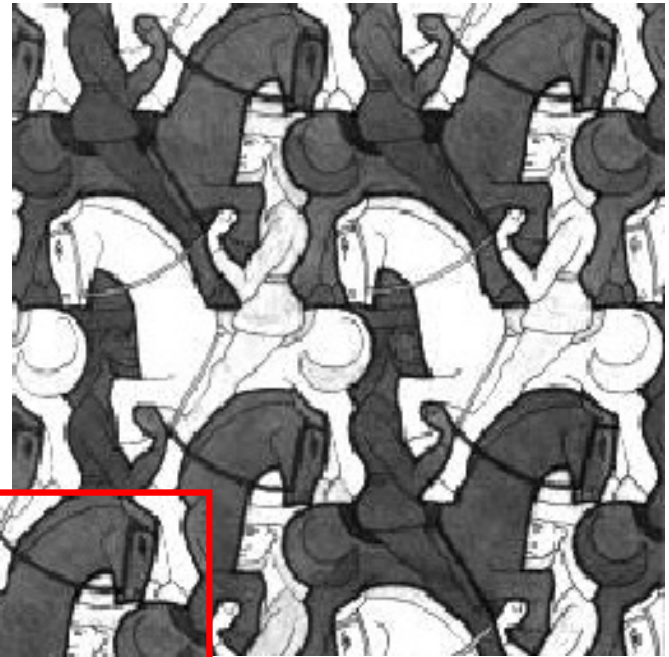
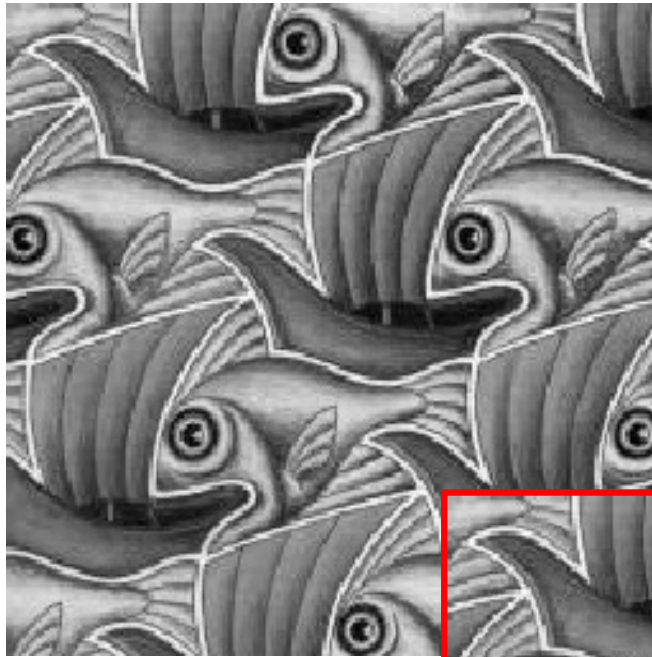
What is happening?

What is/are the issue(s)?

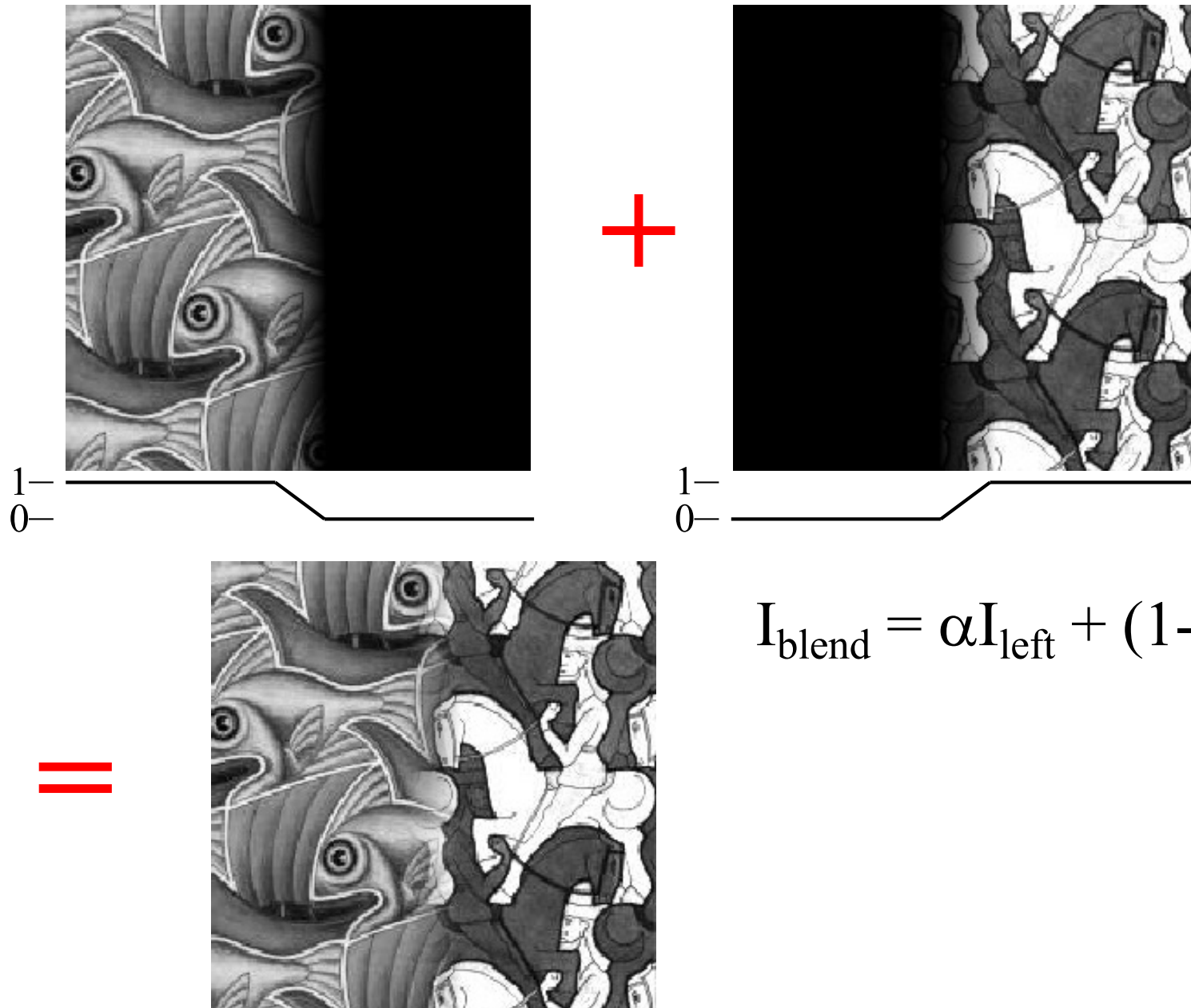


Ideas for solving the issue(s)?

Blending

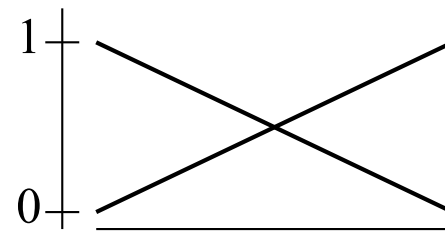
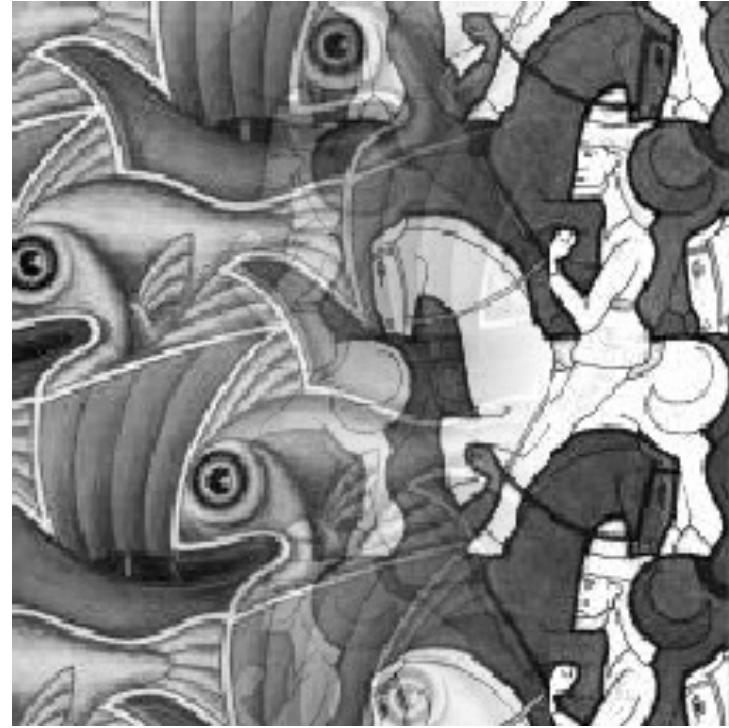
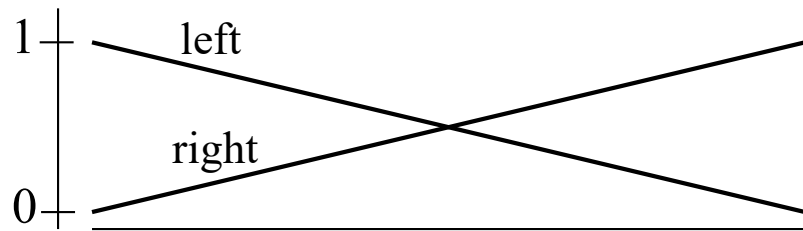


Alpha Blending / Feathering

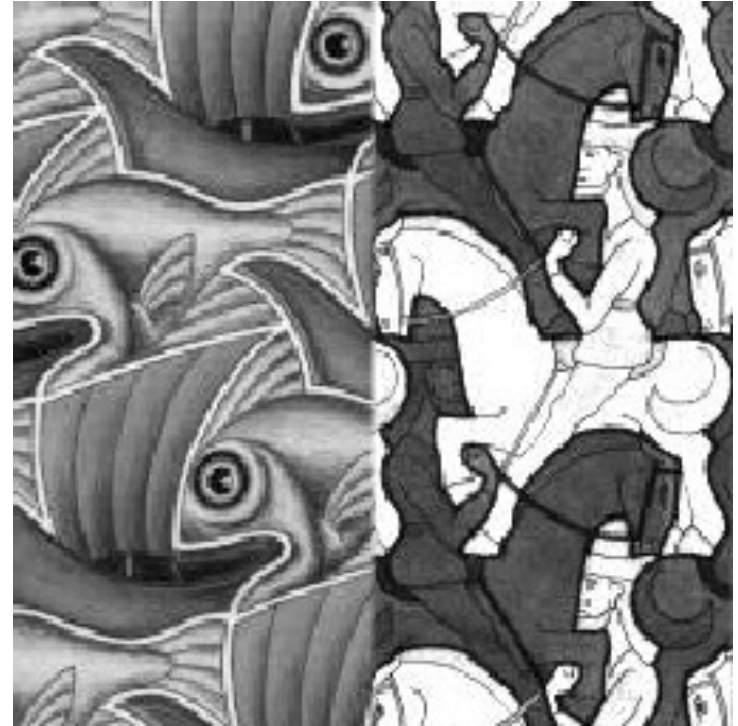
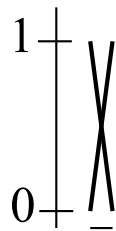


$$I_{\text{blend}} = \alpha I_{\text{left}} + (1-\alpha) I_{\text{right}}$$

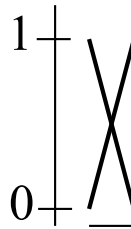
Effect of Window Size



Effect of Window Size



Good Window Size



“Optimal” Window: smooth but not ghosted

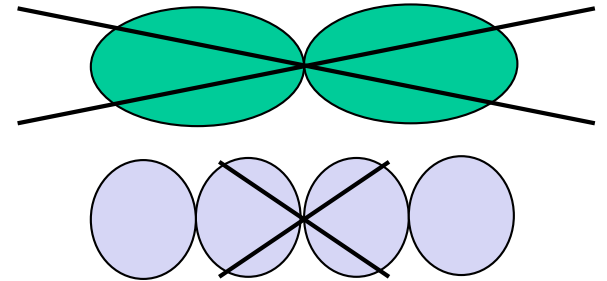
What is the Optimal Window?

To avoid seams

- window = size of largest prominent feature

To avoid ghosting

- window $\leq 2 \times$ size of smallest prominent feature

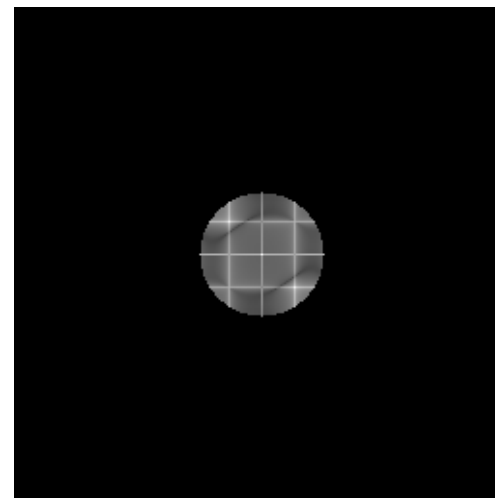


Natural to cast this in the *Fourier domain*

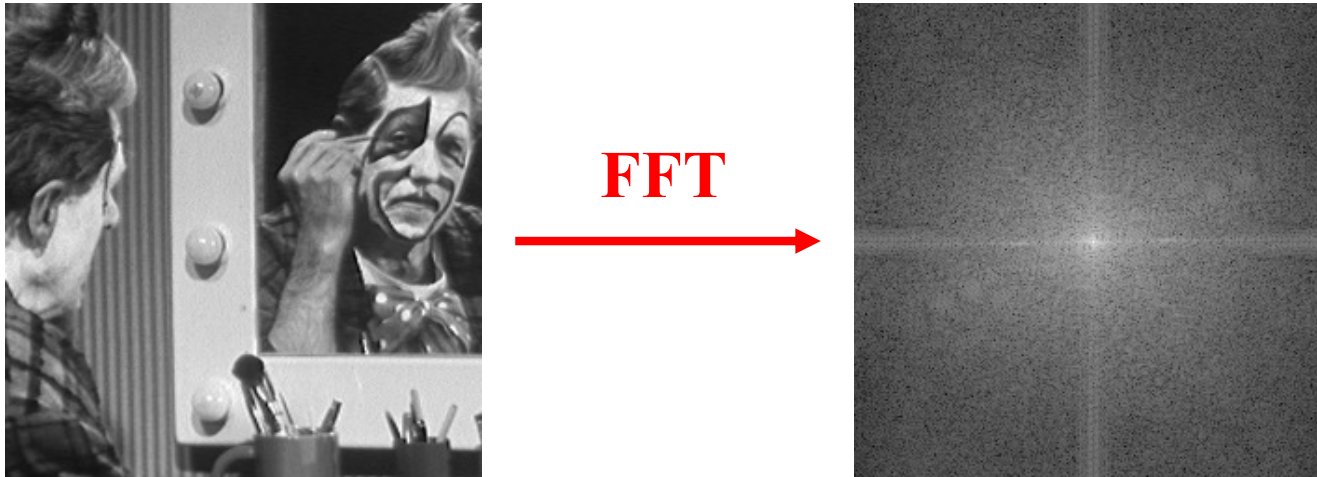
- largest frequency $\leq 2 \times$ size of smallest frequency
- image frequency content should occupy one “octave” (power of two)



FFT
→



What if the Frequency Spread is Wide

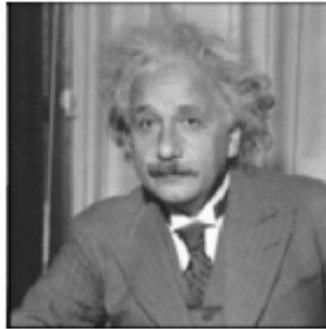
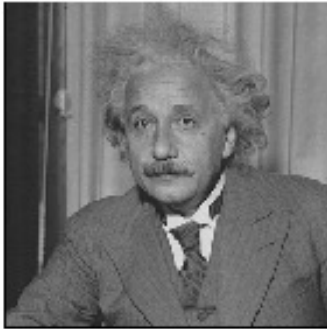


Use a band-pass (Laplacian) Pyramid!

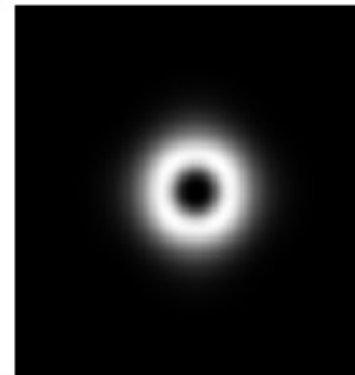
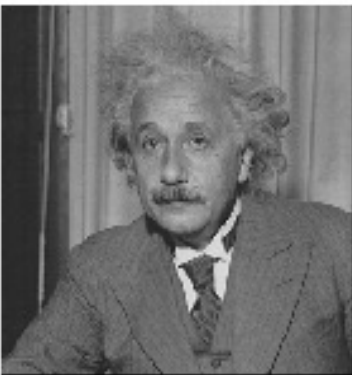
- Split image into set of band-pass images (one octave of frequencies each)
- Blend each level of the pyramid separately
- Collapse the pyramid!

Low-pass, Band-pass, High-pass filters

low-pass:



band-pass:



Low Pass vs. High Pass filtering

Image



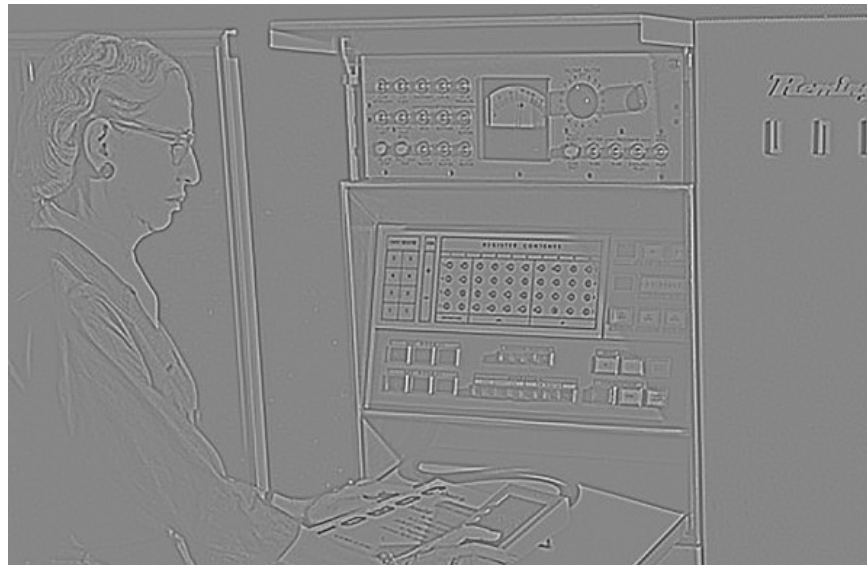
Smoothed



—

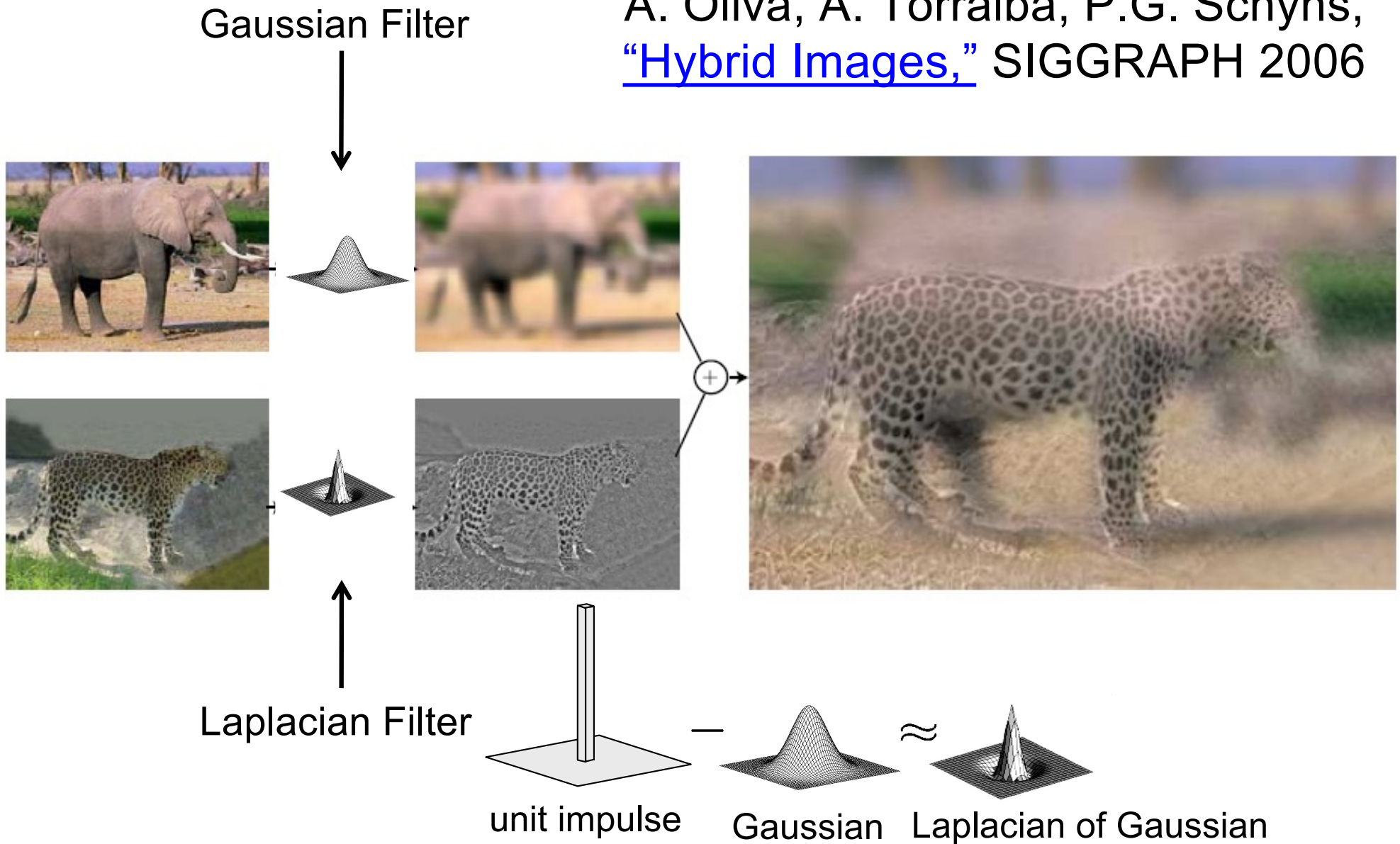
Details

=



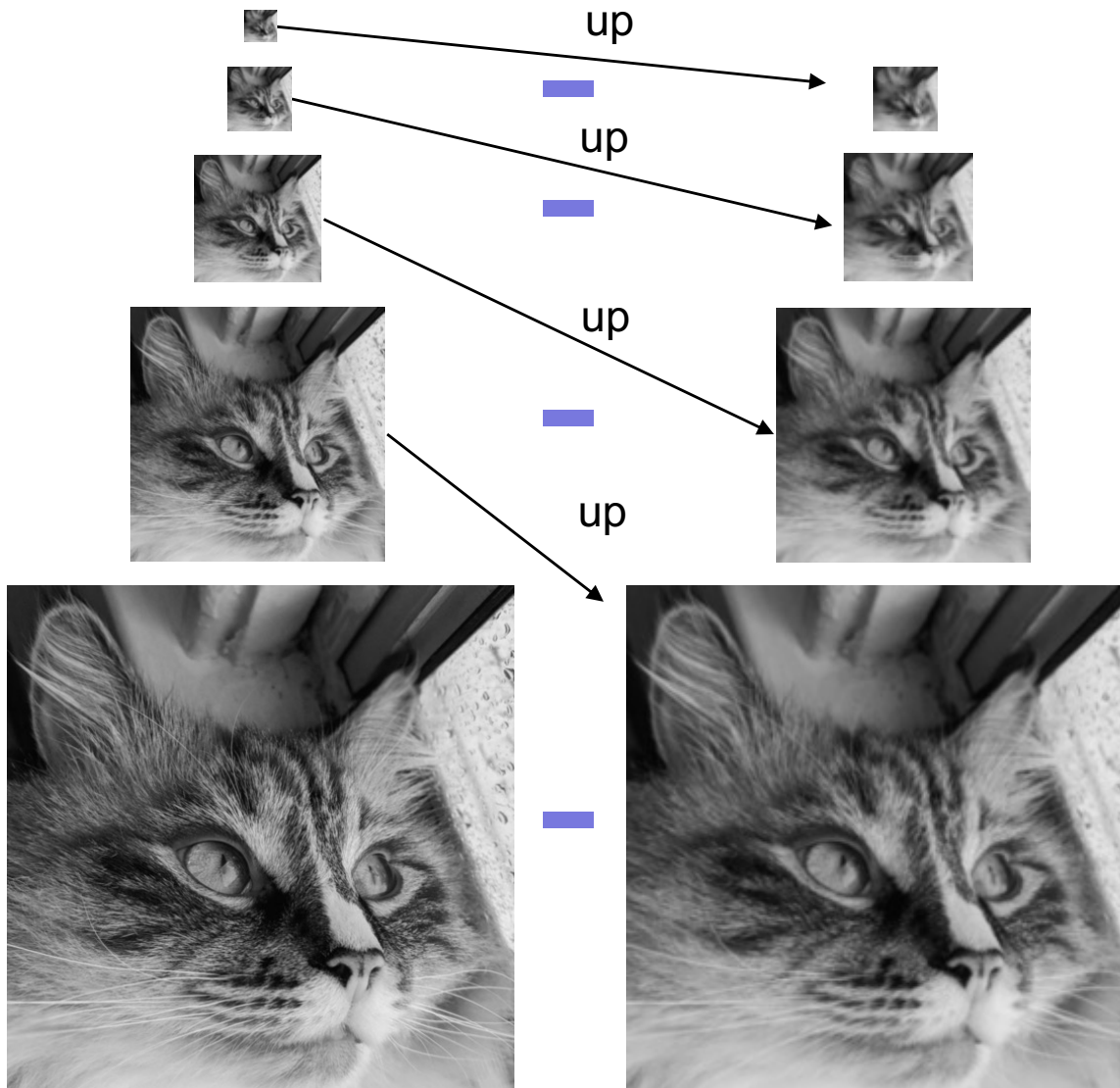
Application: Hybrid Images

A. Oliva, A. Torralba, P.G. Schyns,
[“Hybrid Images,”](#) SIGGRAPH 2006

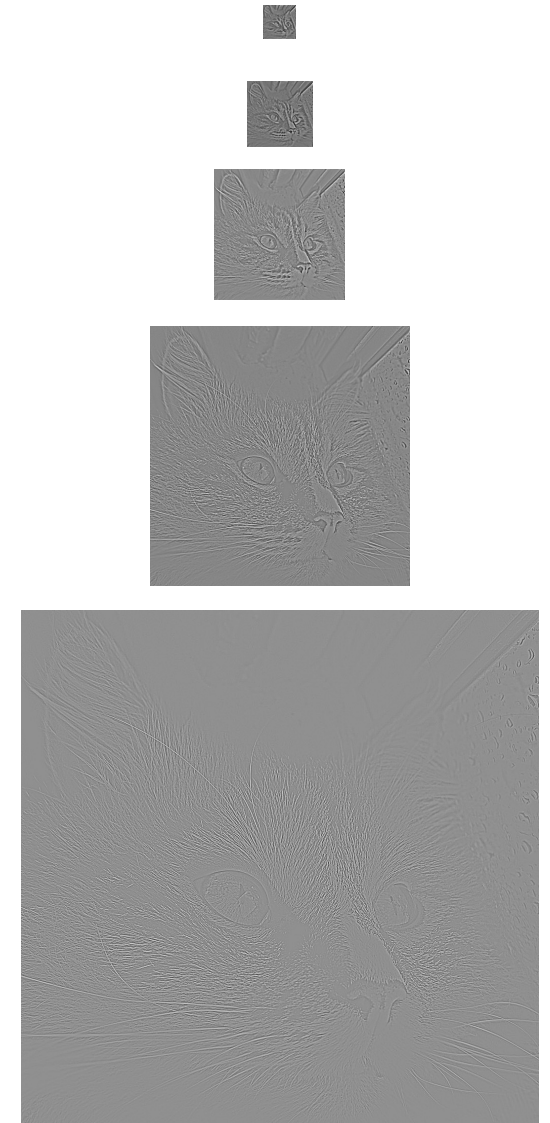


Band-pass filtering in spatial domain

Gaussian Pyramid
(low-pass images) :

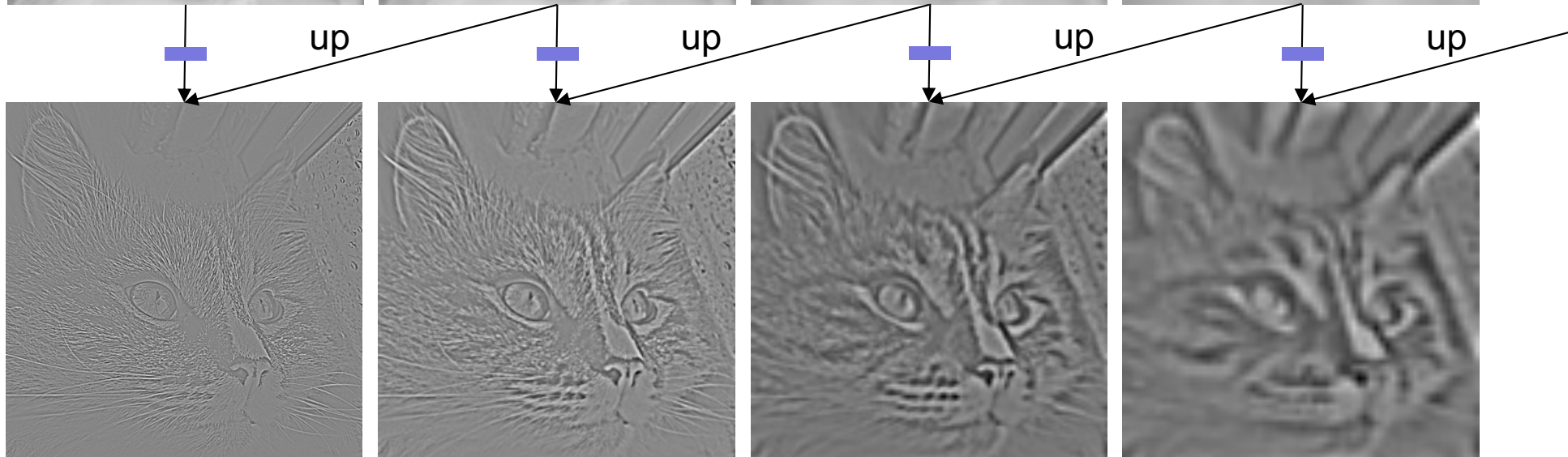


Laplacian Pyramid
(sub-band images)



As a stack

Gaussian Pyramid (low-pass images)

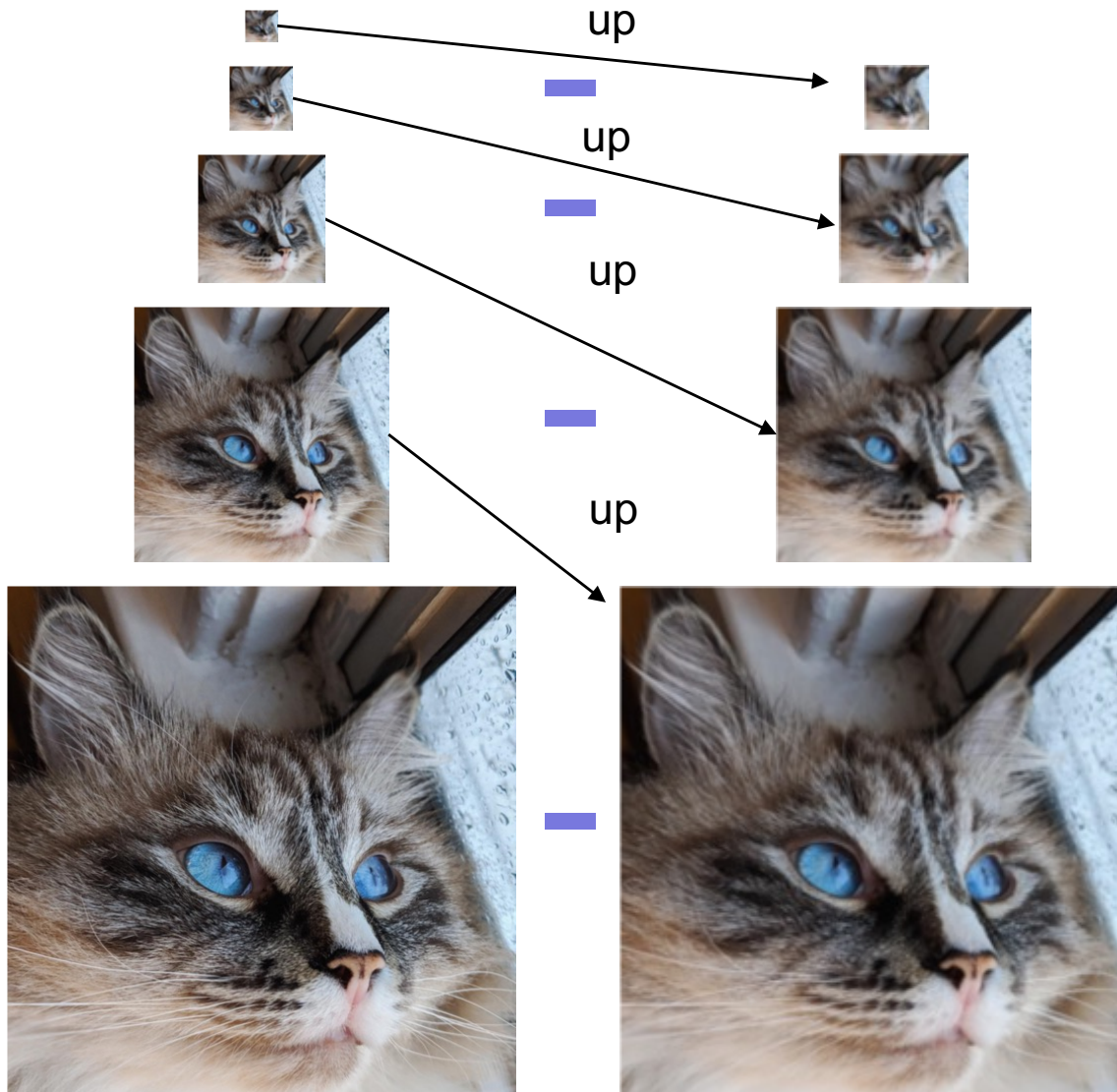


Laplacian Pyramid (sub-band images)

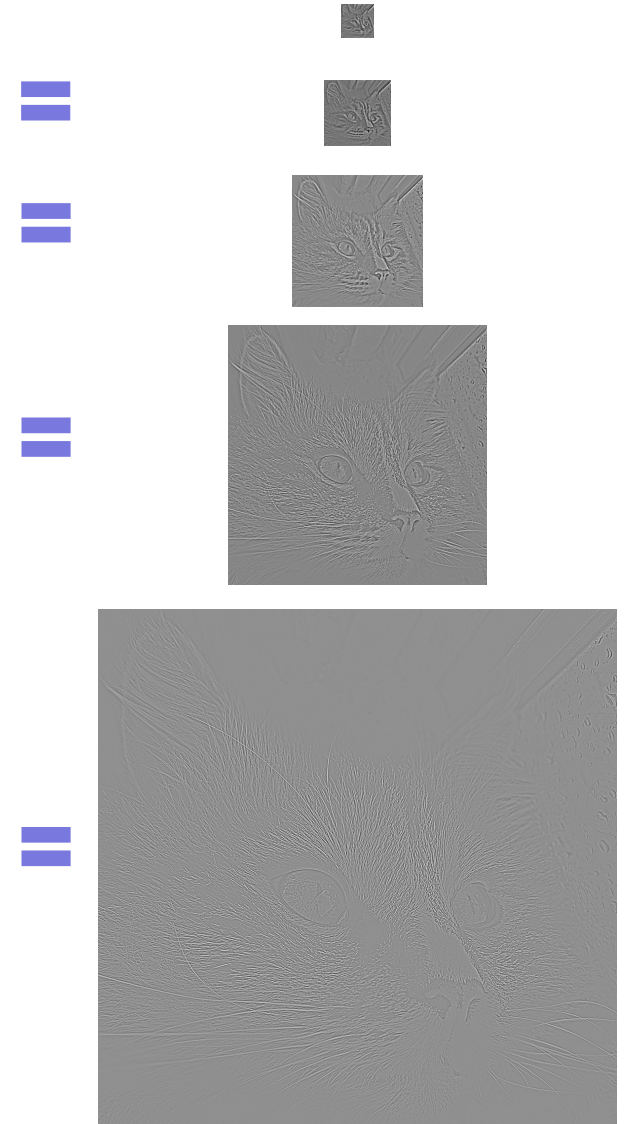
Created from Gaussian pyramid by subtraction

Band-pass filtering in spatial domain

Gaussian Pyramid
(low-pass images) :

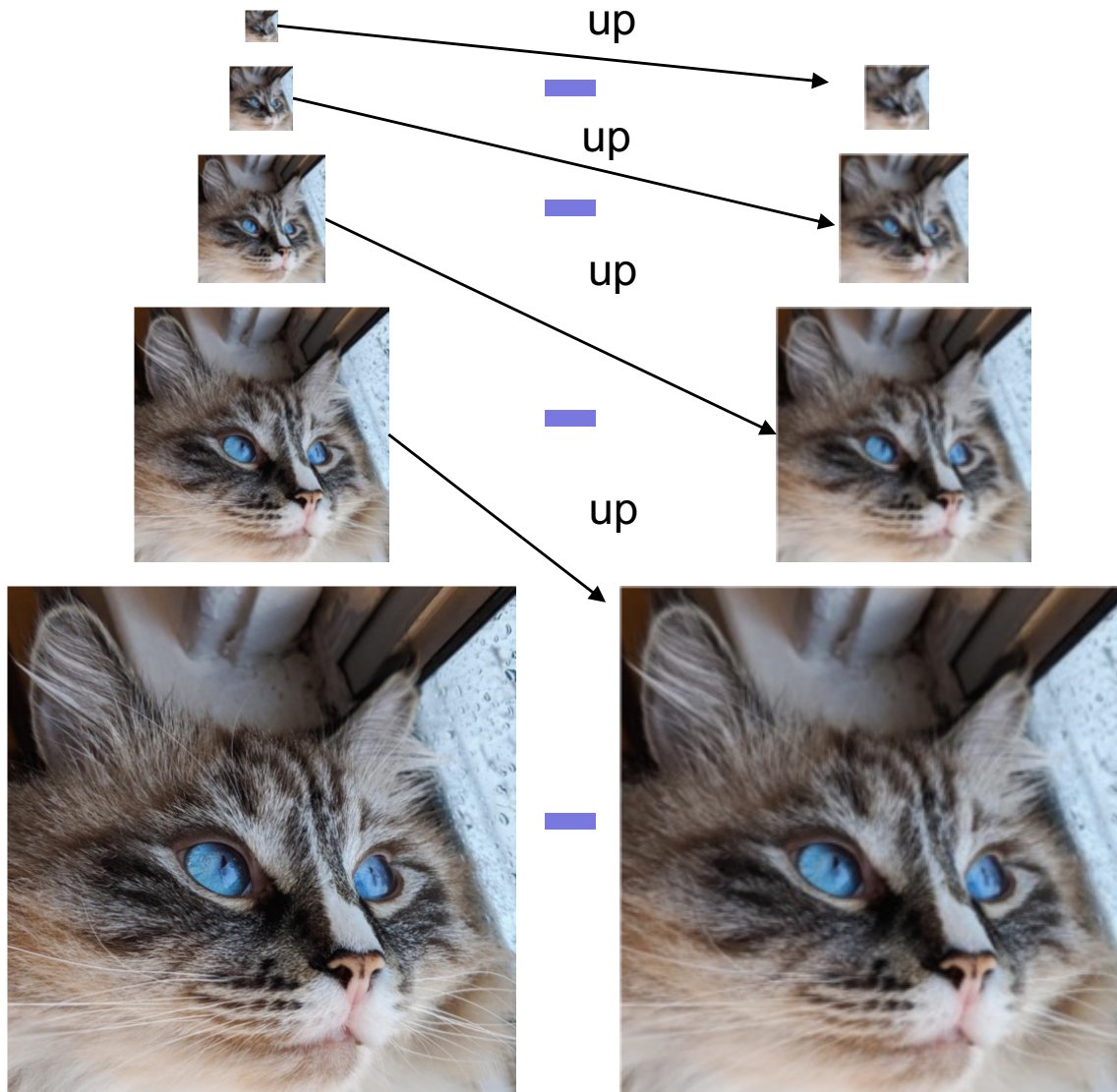


Laplacian Pyramid
(sub-band images)



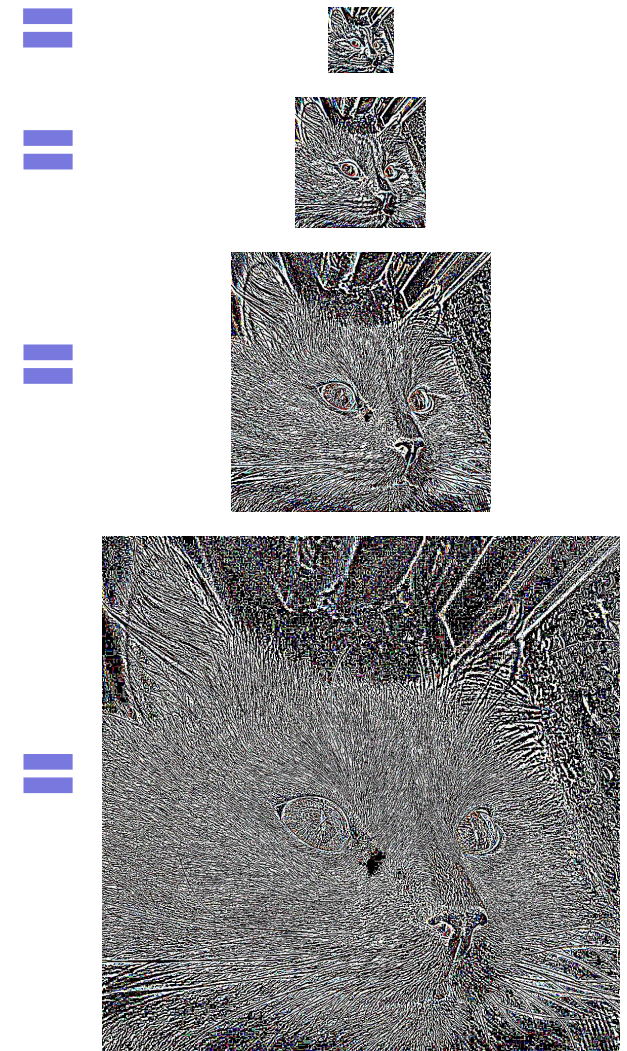
Band-pass filtering in spatial domain

Gaussian Pyramid
(low-pass images)



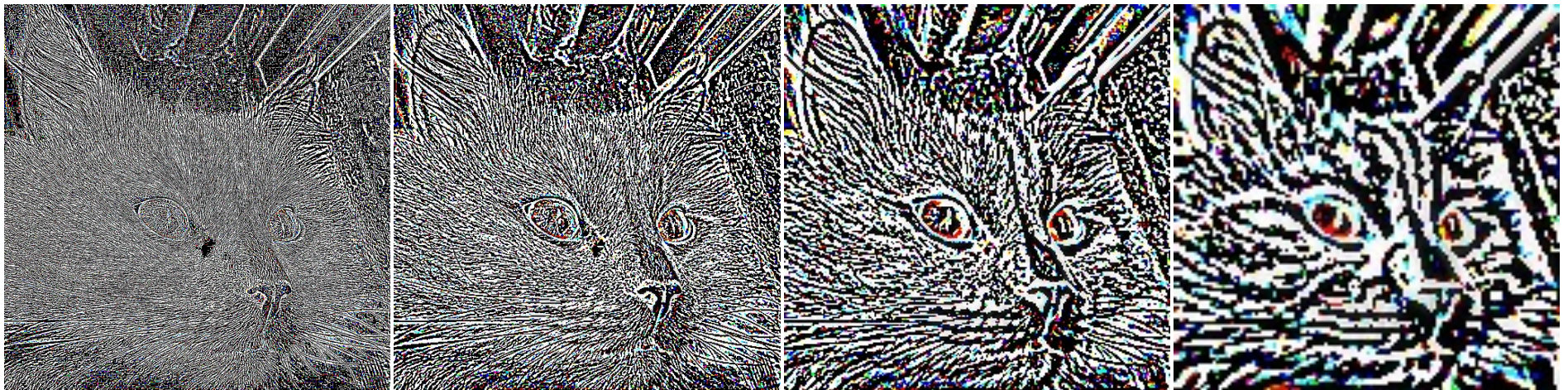
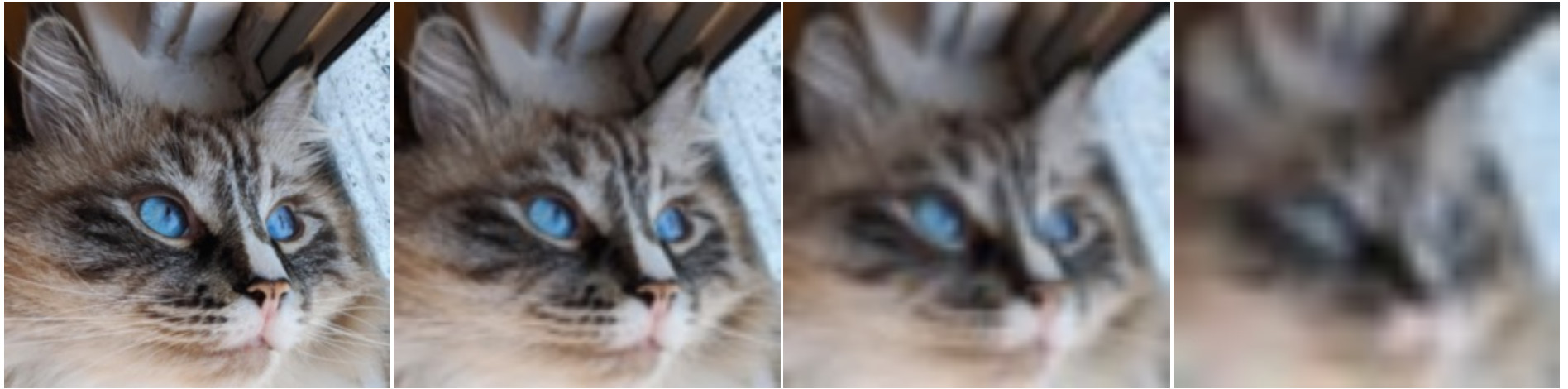
Laplacian Pyramid
(sub-band images)

(contrast enhanced to see detail)



Band-pass filtering in spatial domain

Gaussian Pyramid as a stack



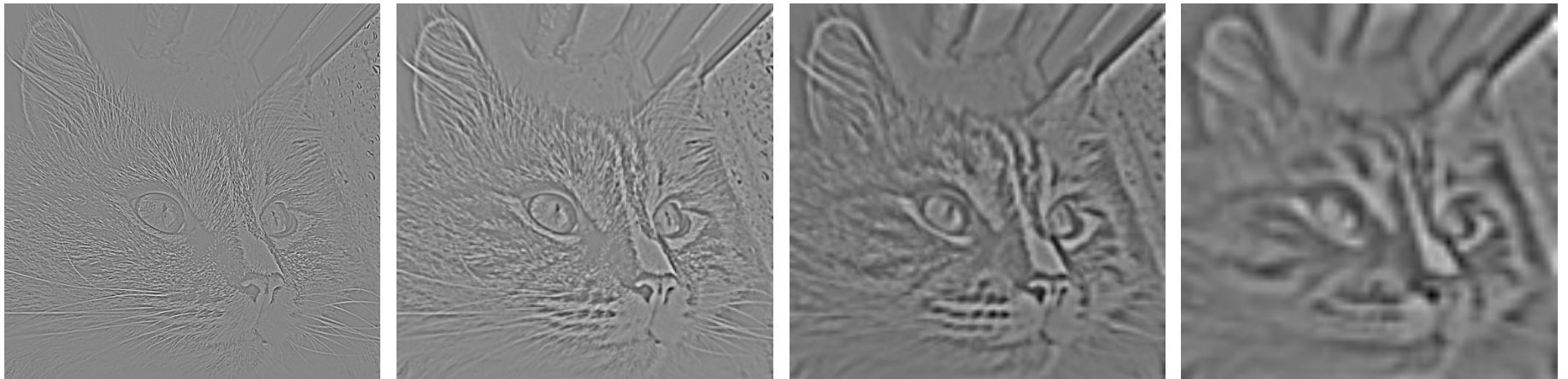
Laplacian Pyramid (sub-band images)

Created from Gaussian pyramid by subtraction

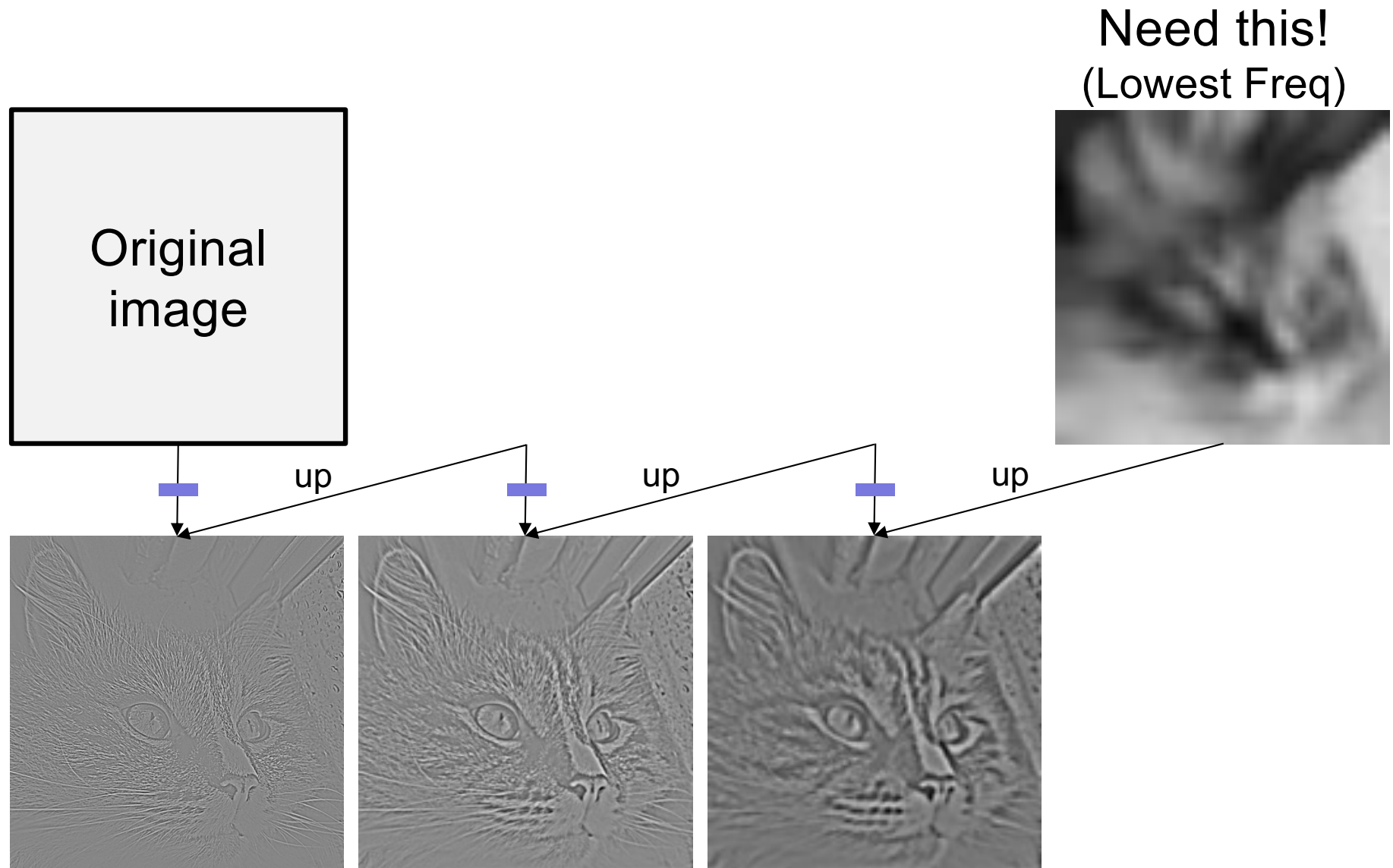
(contrast enhanced
to see detail)

Original Image from Laplacian Pyramid?

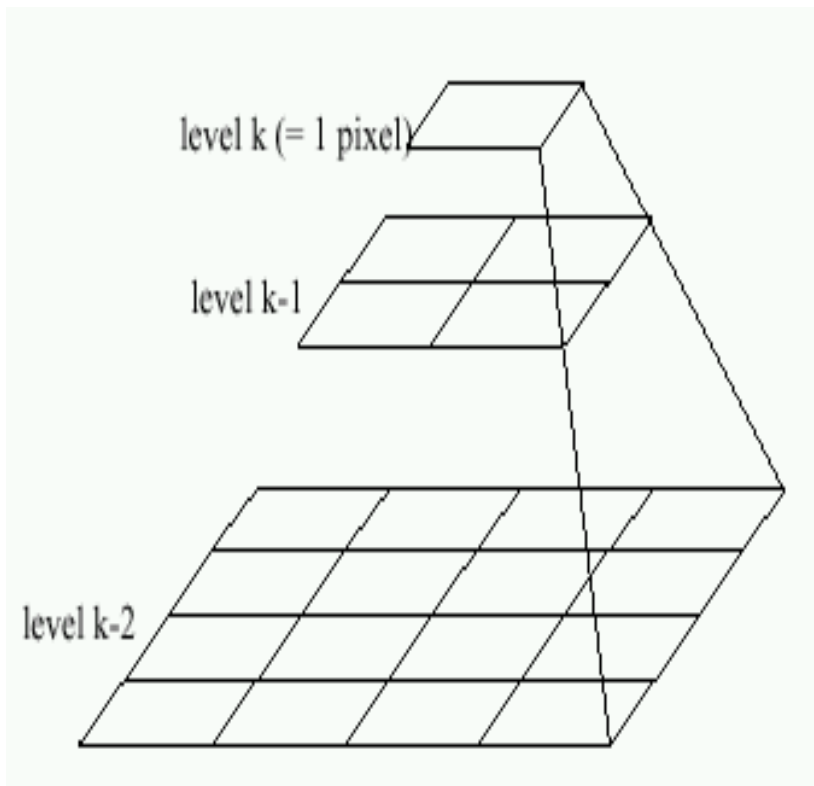
How do we reconstruct (collapse)
this pyramid into the original image?



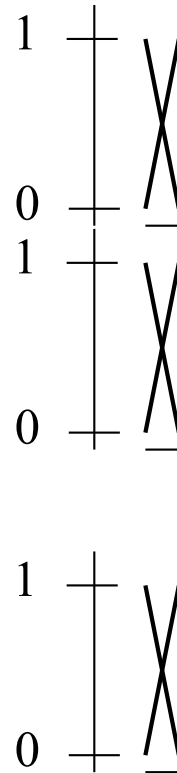
Collapsing Laplacian Pyramid



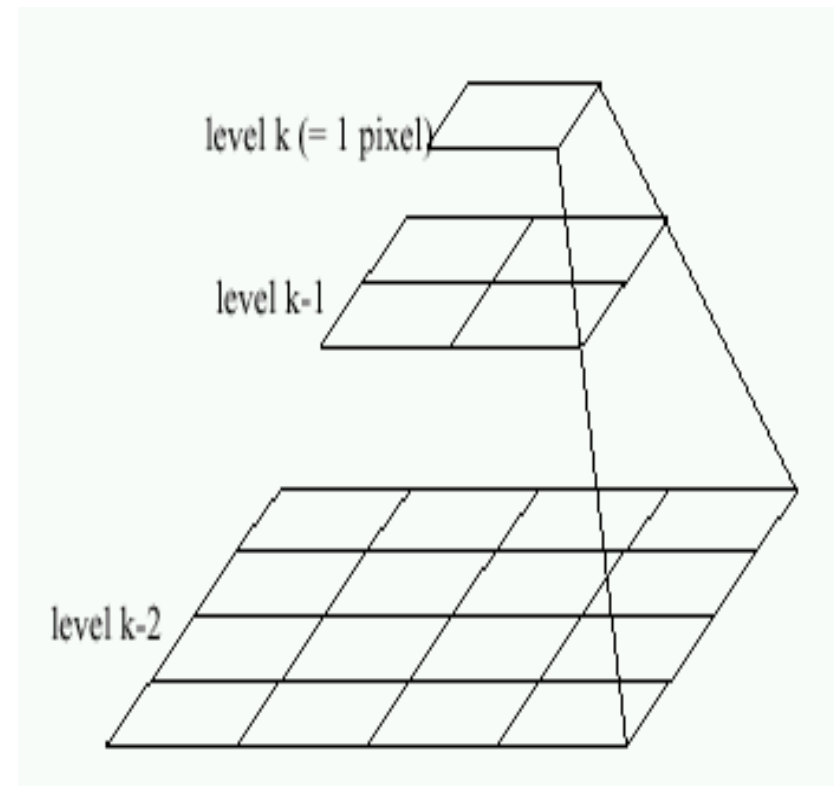
Band-pass Pyramid Blending



Left pyramid

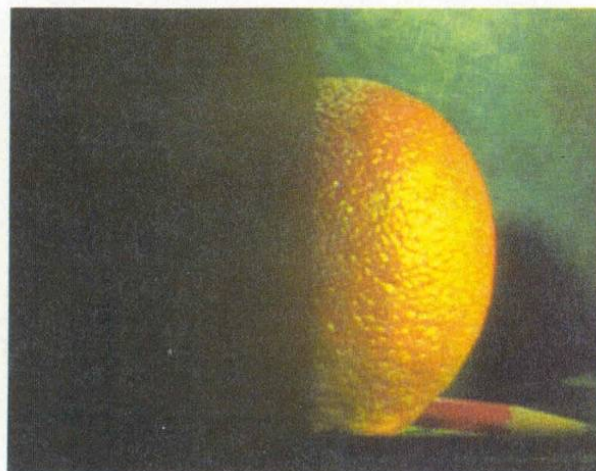
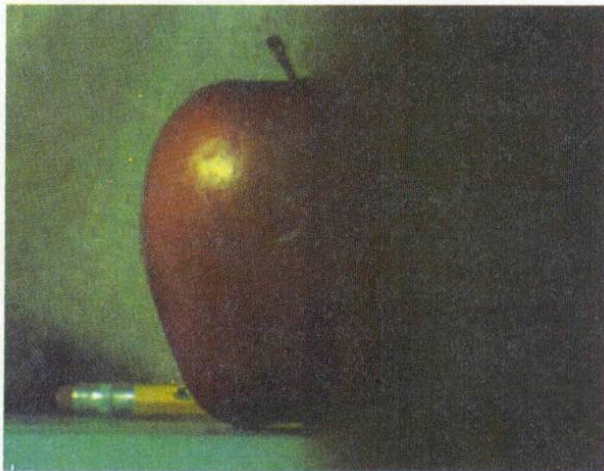
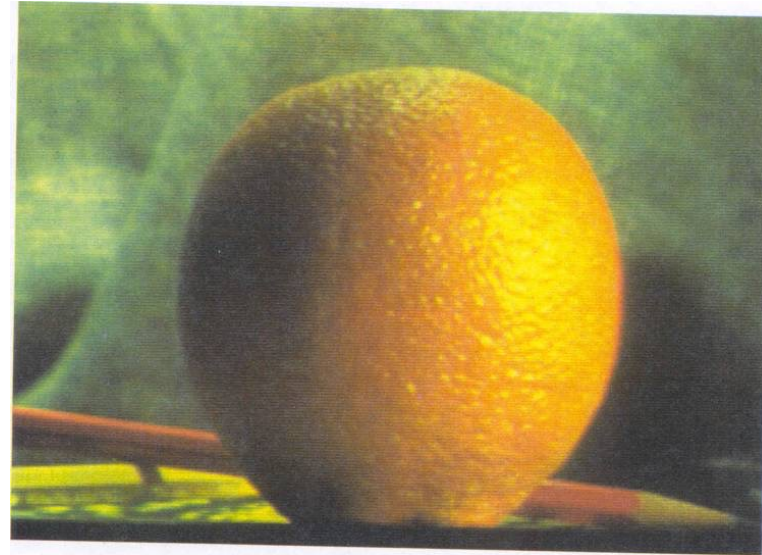
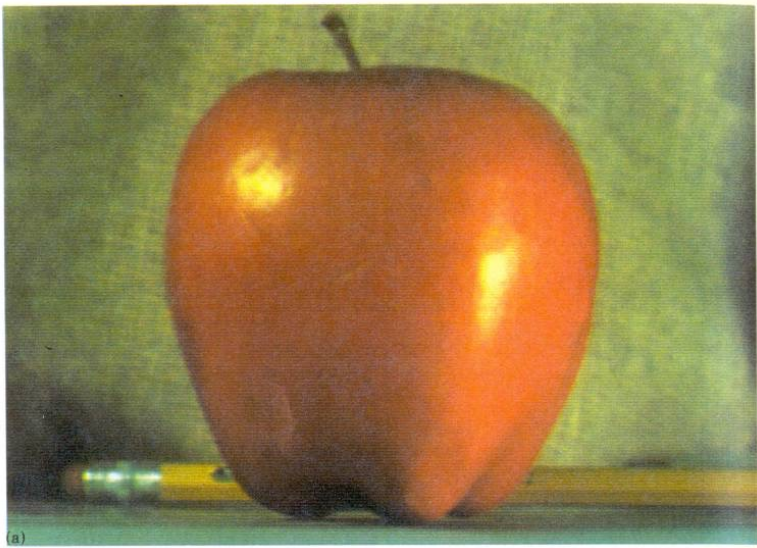


blend



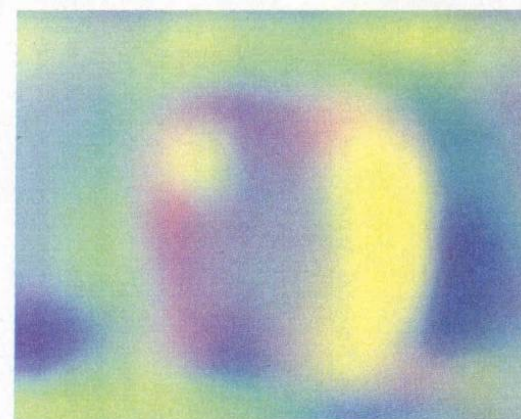
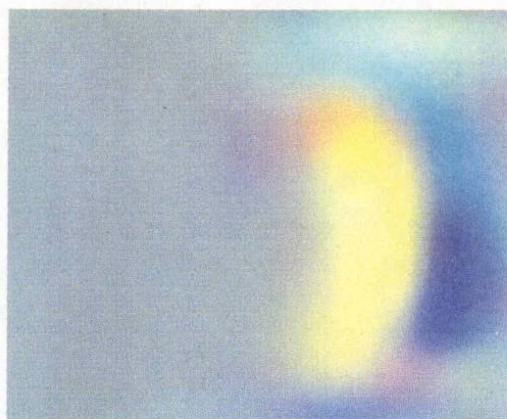
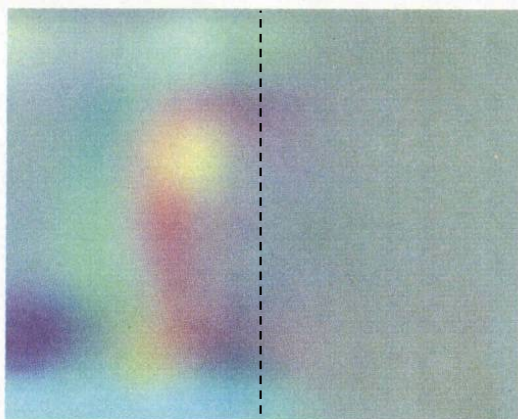
Right pyramid

Pyramid Blending (Burt and Adelson)

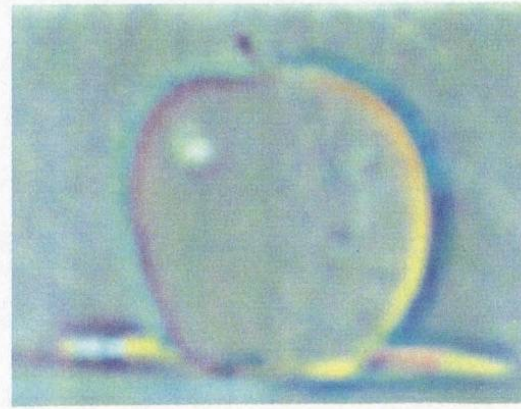
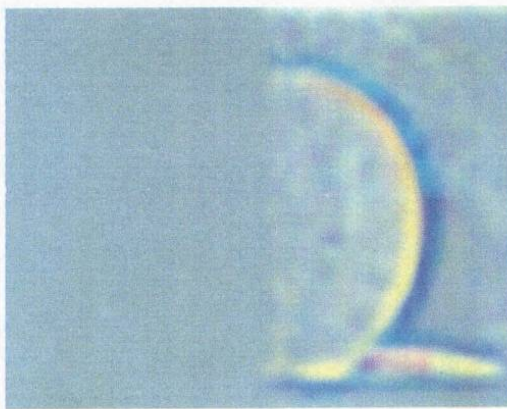
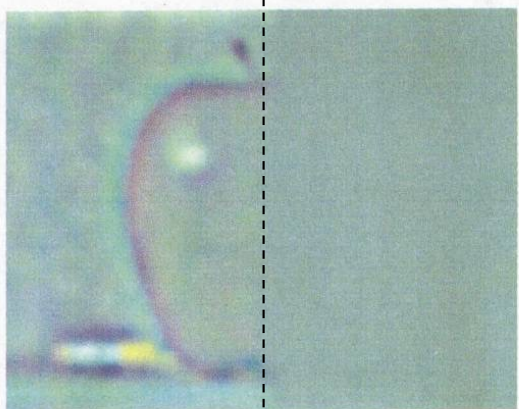


Burt and Adelson (1983), A Multiresolution Spline With Application to Image Mosaics

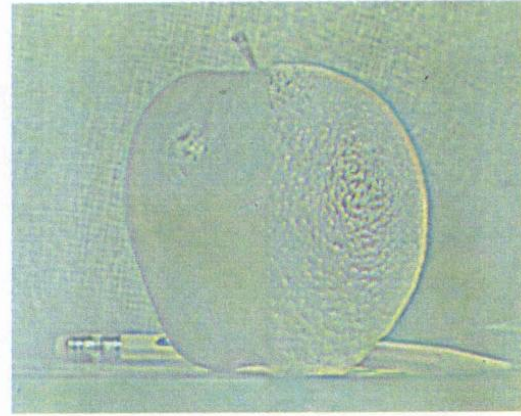
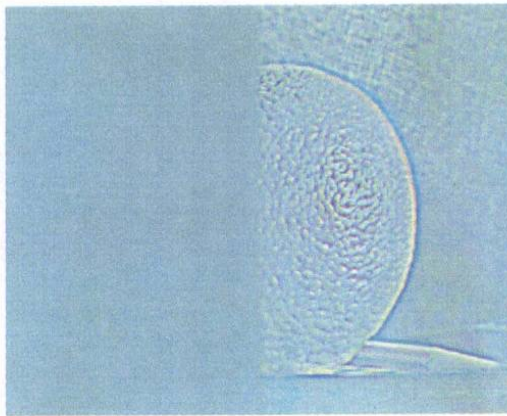
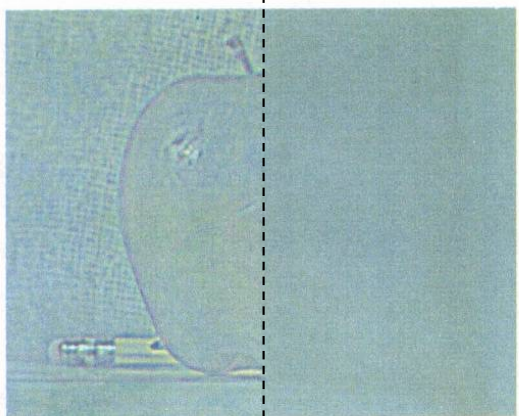
laplacian
level
4



laplacian
level
2



laplacian
level
0

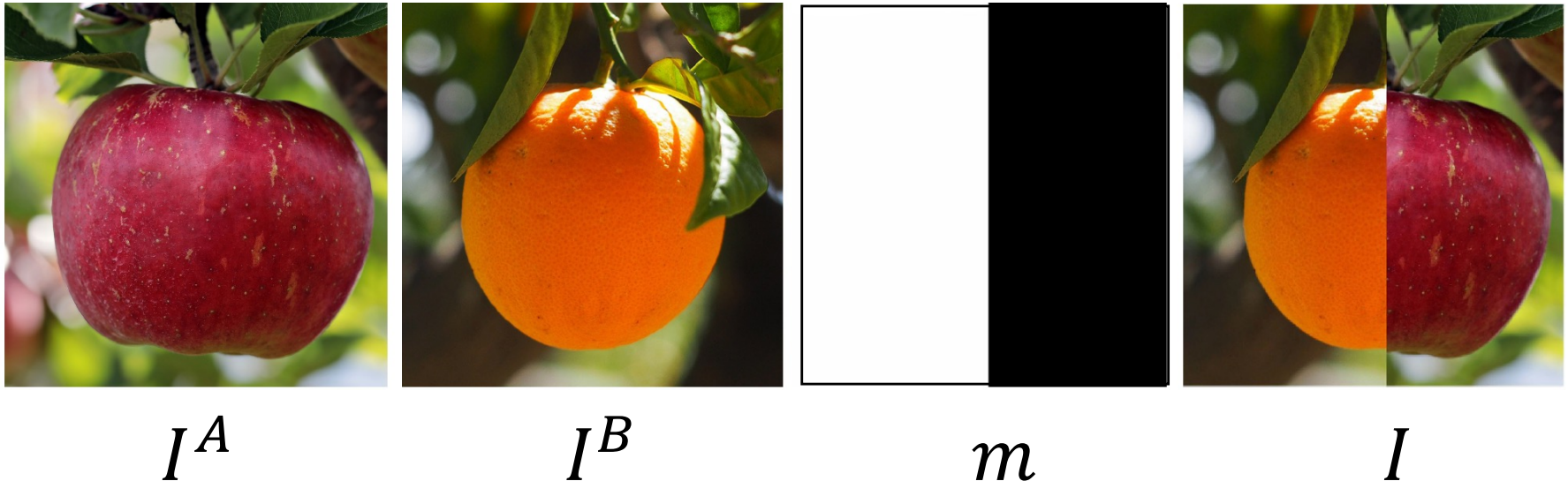


left pyramid

right pyramid

blended pyramid

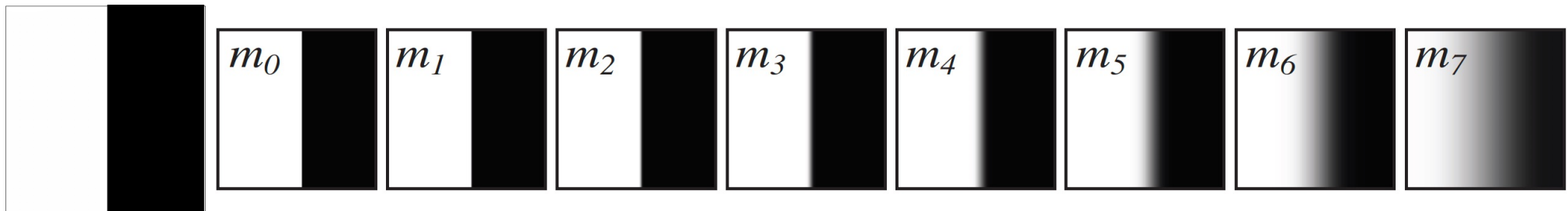
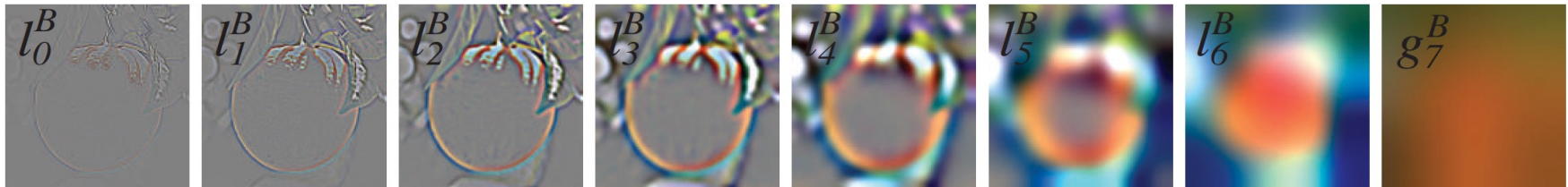
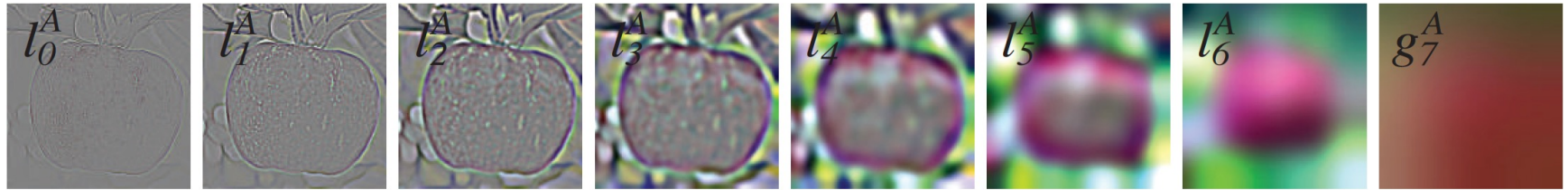
Image Blending with Mask



$$I = I^A * (1 - m) + I^B * m$$

pixel-wise image multiplication and summation

Image Pyramid Blending with Masks



$$l_k = l_k^A * (1 - m_k) + l_k^B * m_k$$

pixel-wise image multiplication and summation

Result



Blending Regions

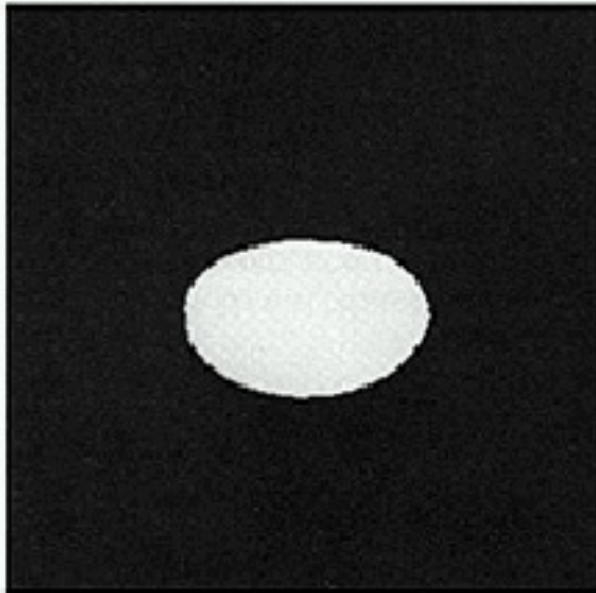


Image Blending with the Laplacian Pyramid

Build Laplacian pyramid for both images: L_A , L_B

Build Gaussian pyramid for mask: G

Build a combined Laplacian pyramid L

Collapse L to obtain the blended image

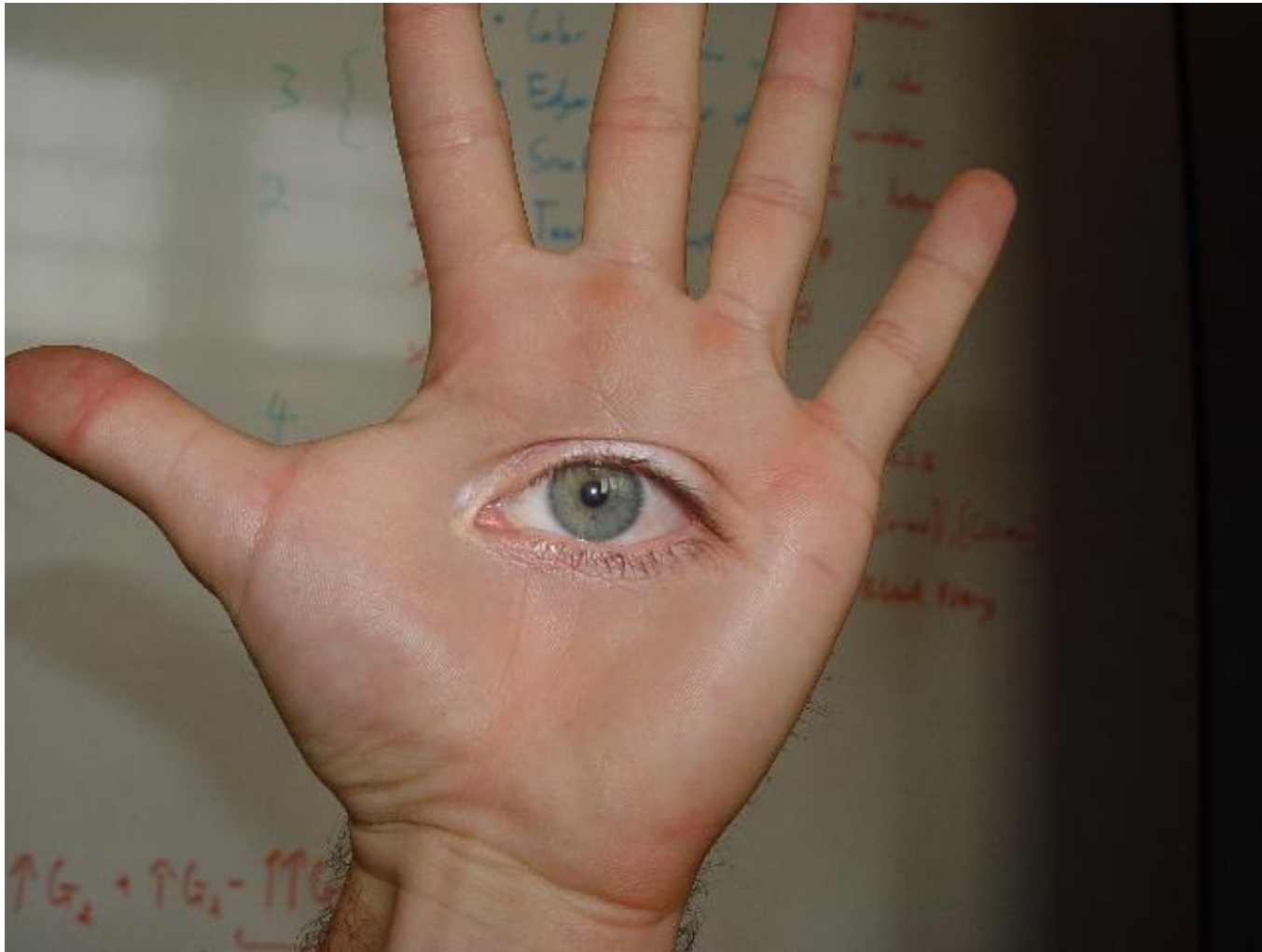
532

IEEE TRANSACTIONS ON COMMUNICATIONS, VOL. COM-31, NO. 4, APRIL 1983

The Laplacian Pyramid as a Compact Image Code

PETER J. BURT, MEMBER, IEEE, AND EDWARD H. ADELSON





© david d martin (Boston College)

Results from this class (fall 2005)



© Chris Cameron

Simplification: Two-band Blending

Brown & Lowe, 2003

- Only use two bands -- high freq. and low freq. – without downsampling
- Blends low freq. smoothly
- Blend high freq. with no smoothing: use binary alpha



Geometrically registered images

2-band “Laplacian Stack” Blending



Geometrically registered images



After two-band blending

2-band “Laplacian Stack” Blending



Low frequency (spatial features > 2 pixels)

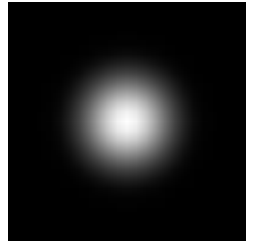


High frequency (spatial features < 2 pixels)

Review: Smoothing vs. derivative filters

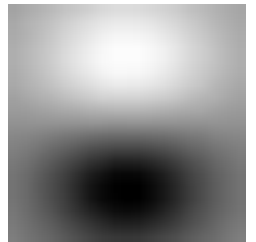
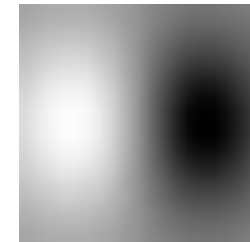
Smoothing filters

- Gaussian: remove “high-frequency” components; “low-pass” filter
- Can the values of a smoothing filter be negative?
- What should the values sum to?
 - **One**: constant regions are not affected by the filter



Derivative filters

- Derivatives of Gaussian
- Can the values of a derivative filter be negative?
- What should the values sum to?
 - **Zero**: no response in constant regions
- High absolute value at points of high contrast

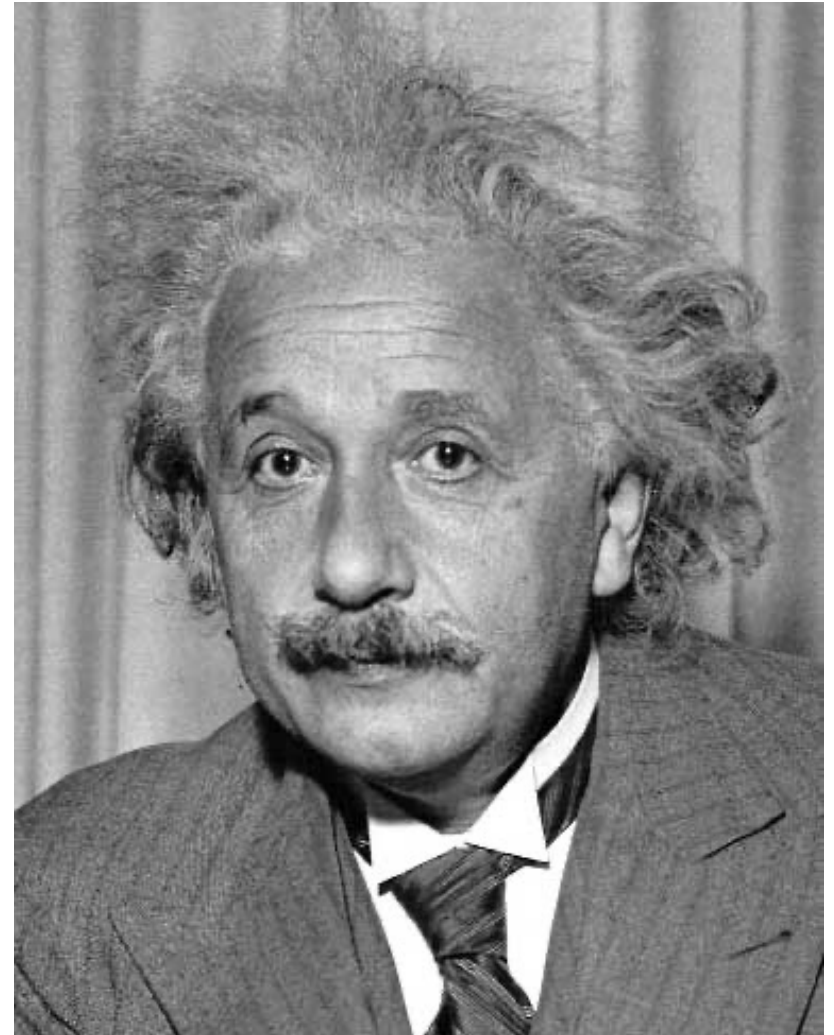


Template matching

Goal: find  in image

Main challenge: What is a good similarity or distance measure between two patches?

- Correlation
- Zero-mean correlation
- Sum Square Difference
- Normalized Cross Correlation



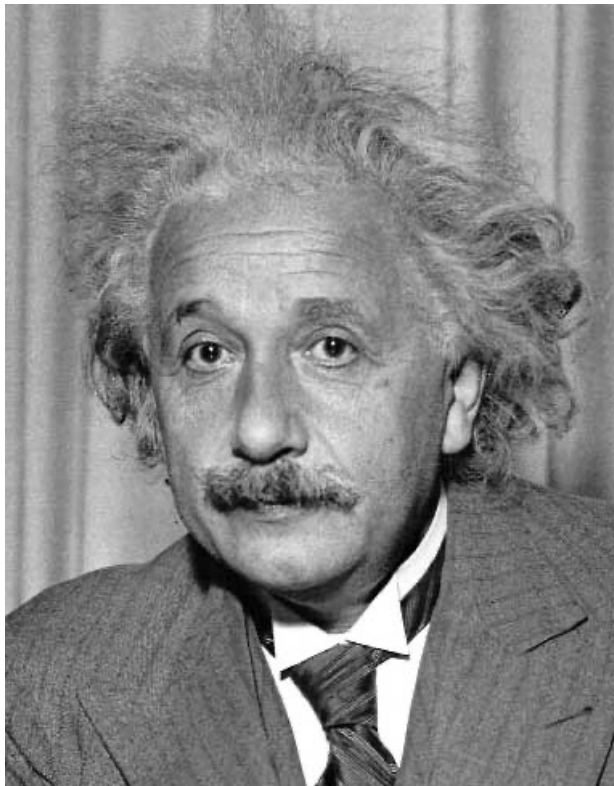
Matching with filters

Goal: find  in image

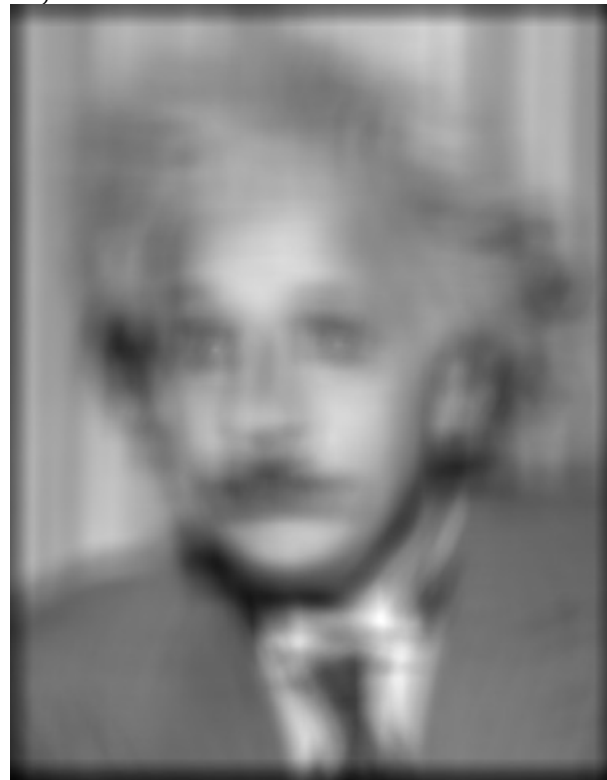
Method 0: filter the image with eye patch

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k, n+l]$$

f = image
g = filter



Input



Filtered Image

What went wrong?

Side by Derek Hoiem

Matching with filters

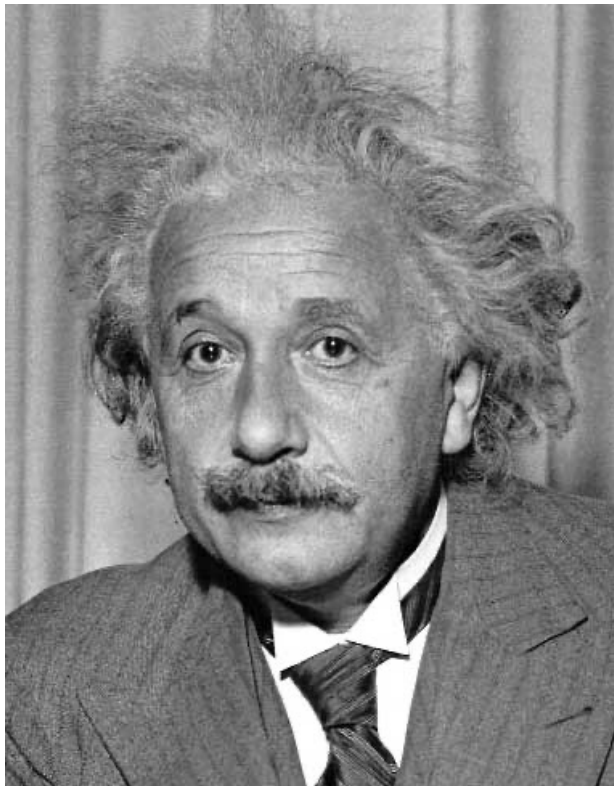
Goal: find  in image

f = image
g = filter

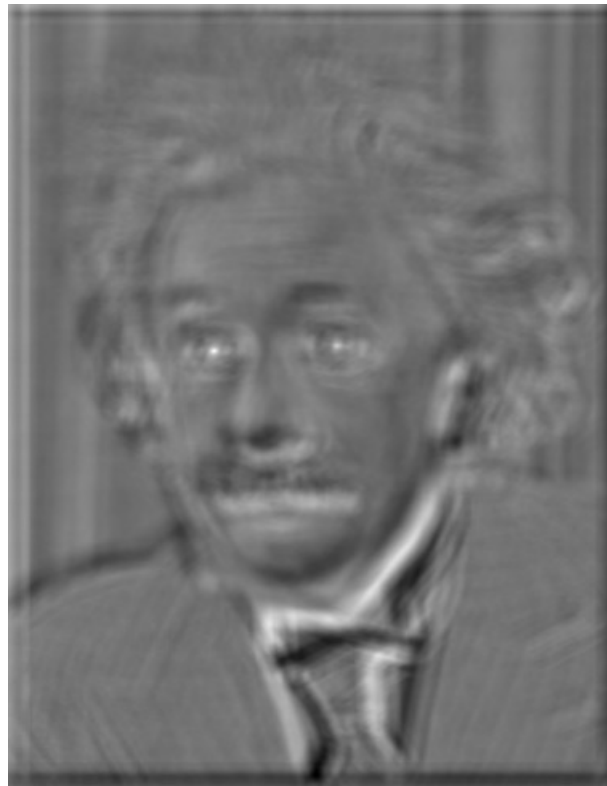
Method 1: filter the image with zero-mean eye

$$h[m, n] = \sum_{k, l} (g[k, l] - \bar{g})(f[m + k, n + l])$$

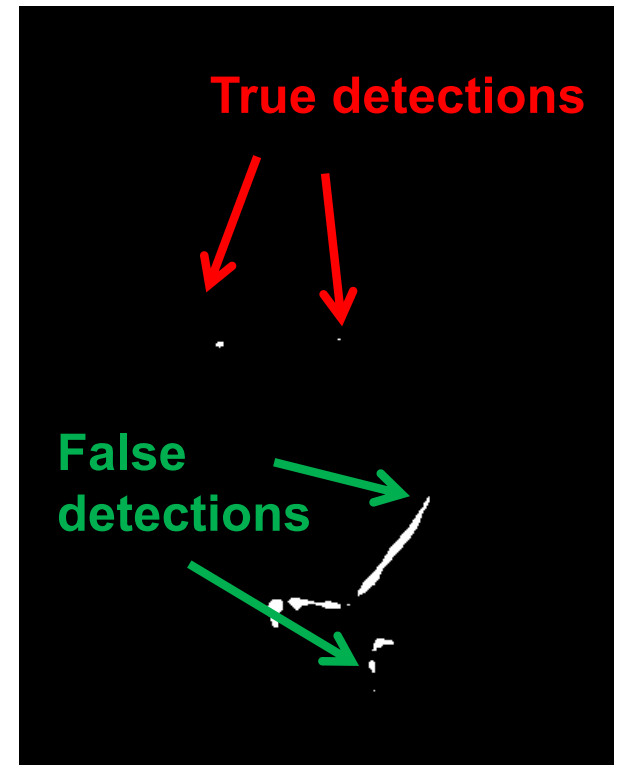
← mean of g



Input



Filtered Image (scaled)



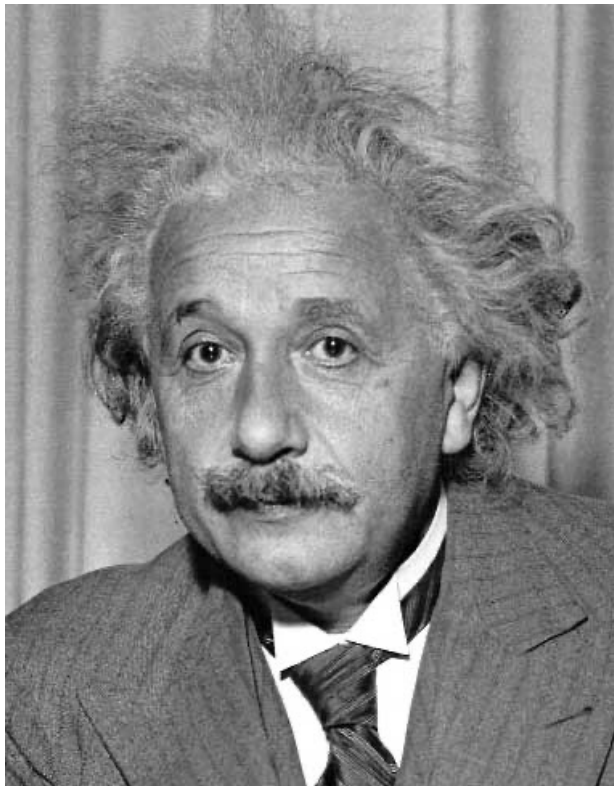
Thresholded Image

Matching with filters

Goal: find  in image

Method 2: SSD (L2)

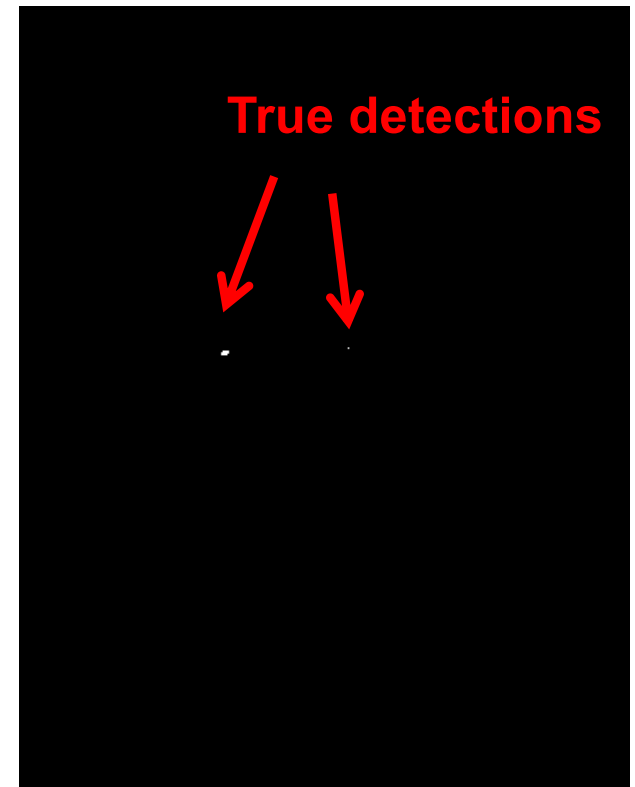
$$h[m,n] = \sum_{k,l} (g[k,l] - f[m+k,n+l])^2$$



Input



1- sqrt(SSD)



Thresholded Image

Matching with filters

Can SSD be implemented with linear filters?

$$h[m,n] = \sum_{k,l} (g[k,l] - f[m+k,n+l])^2$$

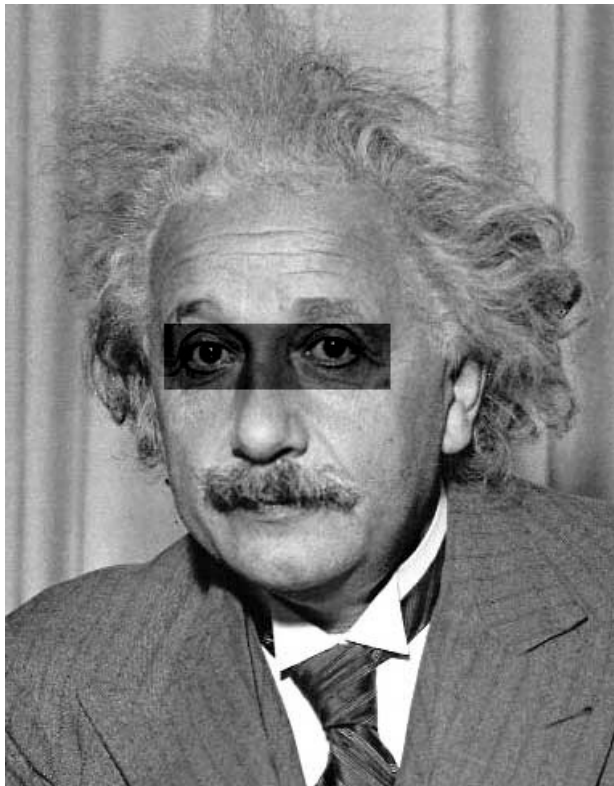
Matching with filters

Goal: find  in image

What's the potential
downside of SSD?

Method 2: SSD

$$h[m,n] = \sum_{k,l} (g[k,l] - f[m+k,n+l])^2$$



Input



1- sqrt(SSD)

Side by Derek Hoiem

Matching with filters

Goal: find  in image

Method 3: Normalized cross-correlation

mean template

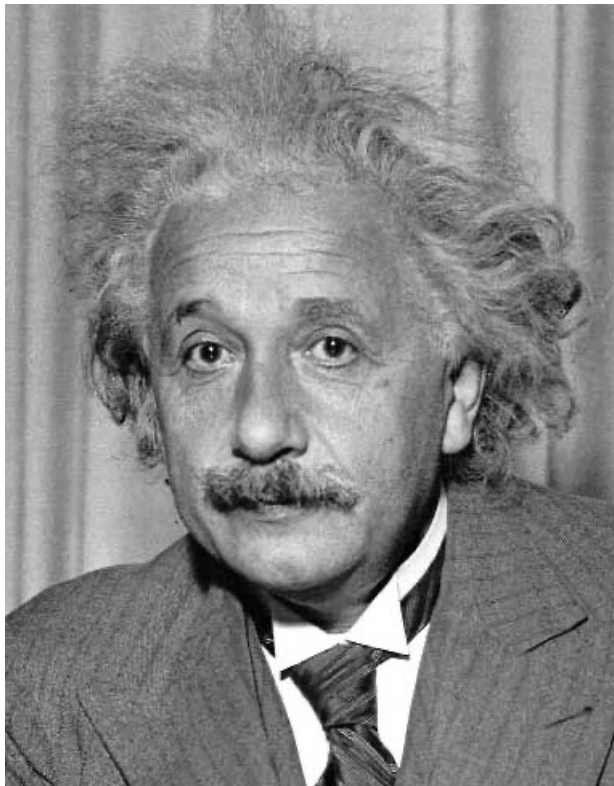
mean image patch

$$h[m,n] = \frac{\sum_{k,l} (g[k,l] - \bar{g})(f[m+k, n+l] - \bar{f}_{m,n})}{\left(\sum_{k,l} (g[k,l] - \bar{g})^2 \sum_{k,l} (f[m+k, n+l] - \bar{f}_{m,n})^2 \right)^{0.5}}$$

Matching with filters

Goal: find  in image

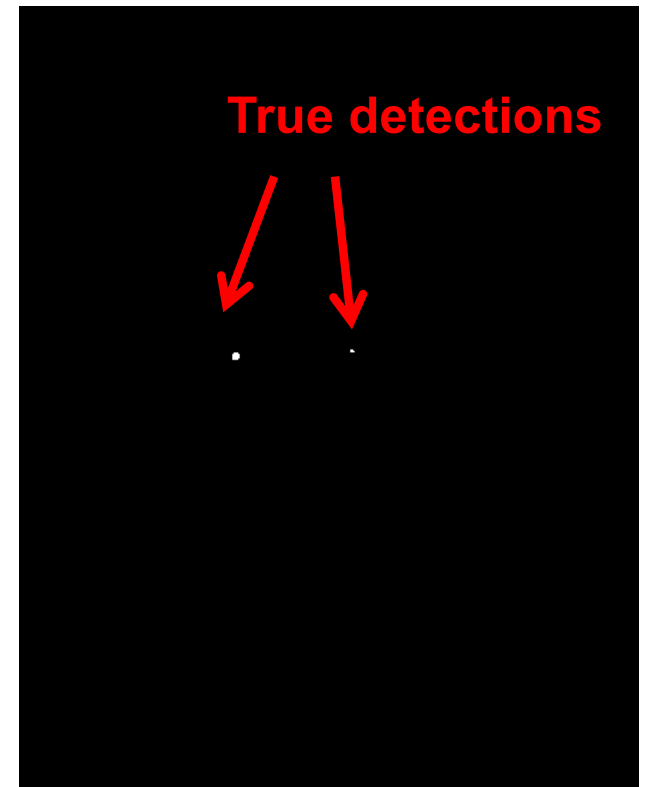
Method 3: Normalized cross-correlation



Input



Normalized X-Correlation

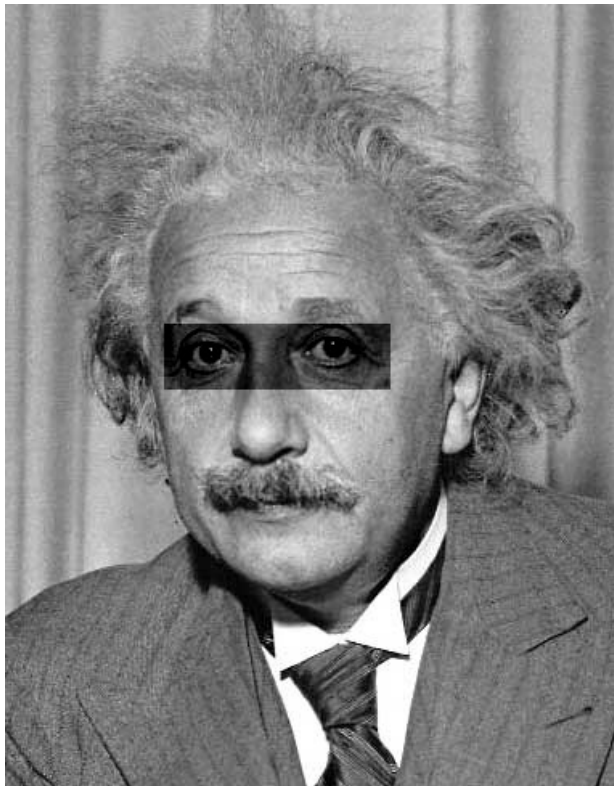


Thresholded Image

Matching with filters

Goal: find  in image

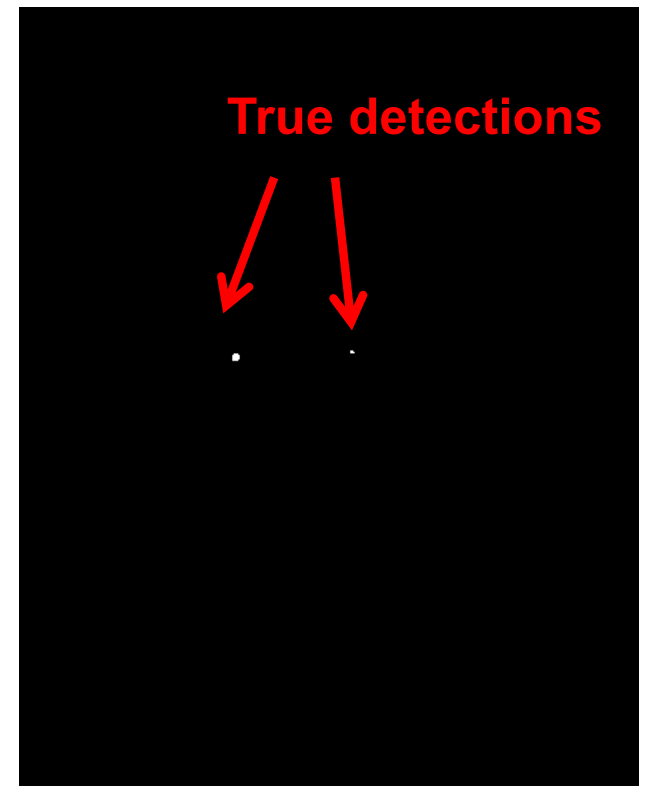
Method 3: Normalized cross-correlation



Input



Normalized X-Correlation



Thresholded Image

Q: What is the best method to use?

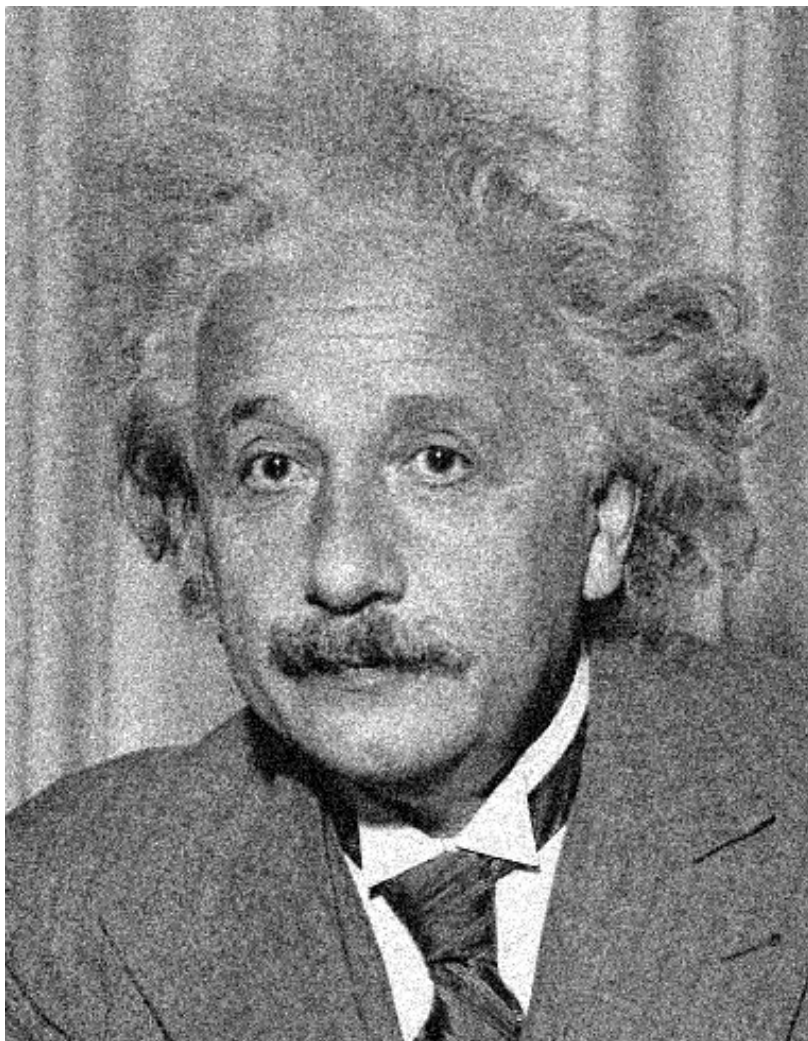
A: Depends

Zero-mean filter: fastest but not a great matcher

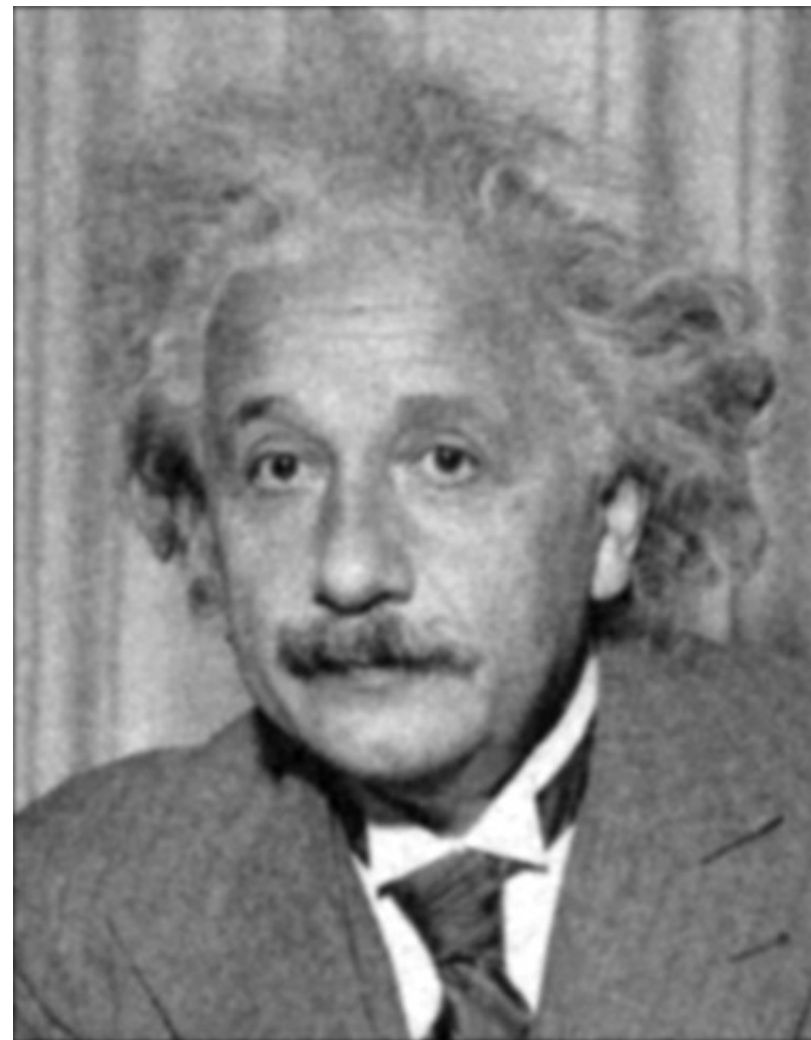
SSD: next fastest, sensitive to overall intensity

Normalized cross-correlation: slowest, invariant to local average intensity and contrast

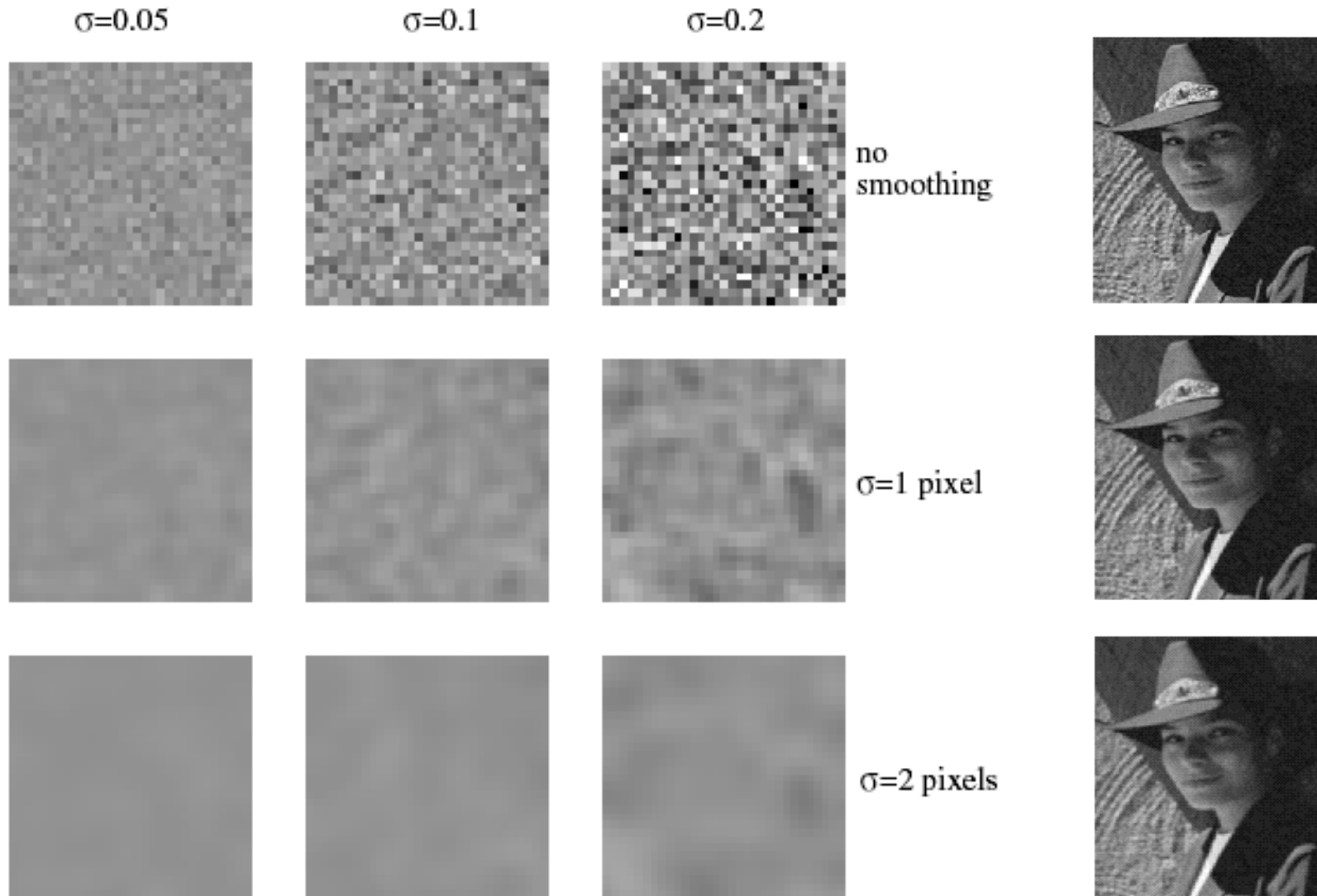
Denoising



Additive Gaussian Noise



Reducing Gaussian noise



Smoothing with larger standard deviations suppresses noise, but also blurs the image

Reducing salt-and-pepper noise by Gaussian smoothing

3x3



5x5

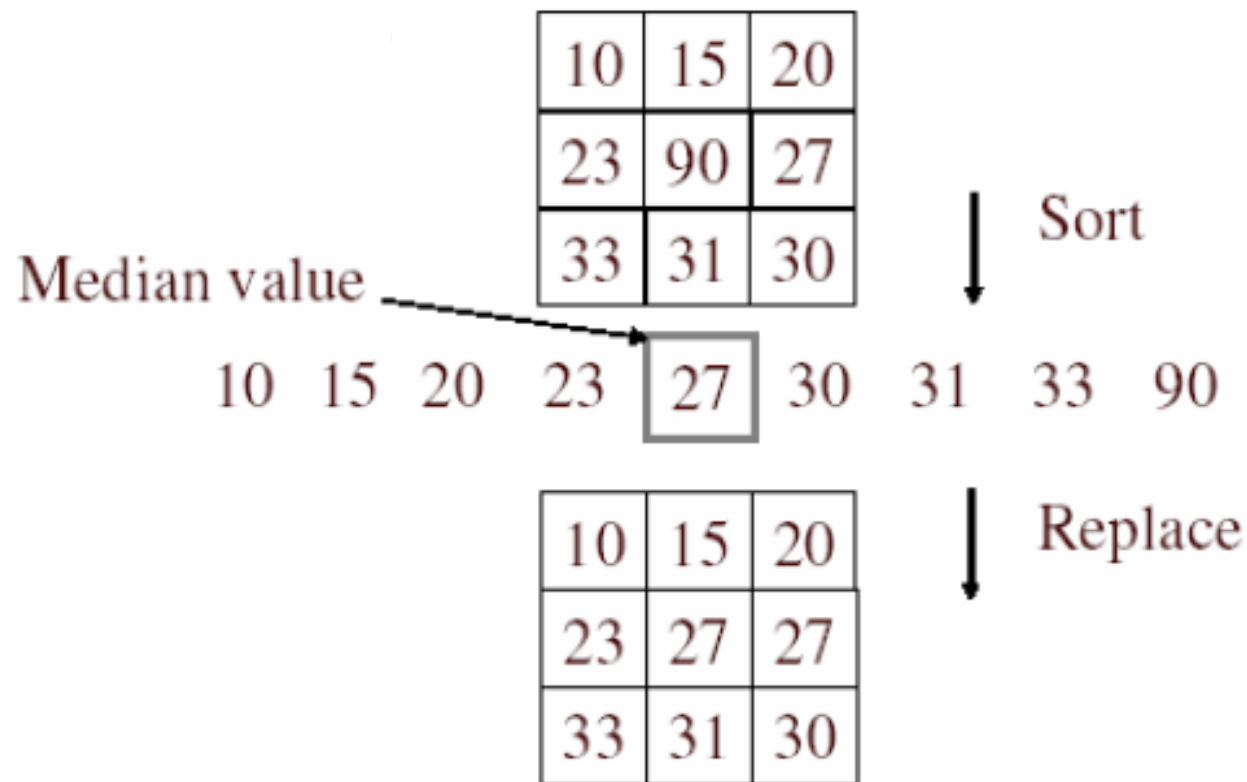


7x7



Alternative idea: Median filtering

A **median filter** operates over a window by selecting the median intensity in the window



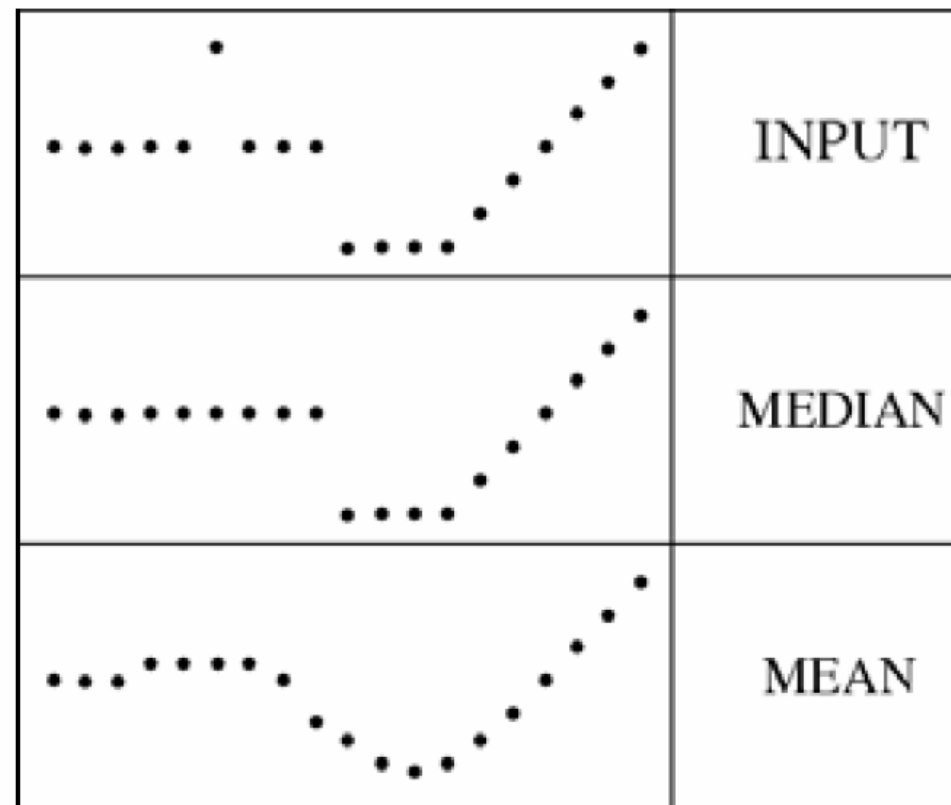
- Is median filtering linear?

Median filter

What advantage does median filtering have over Gaussian filtering?

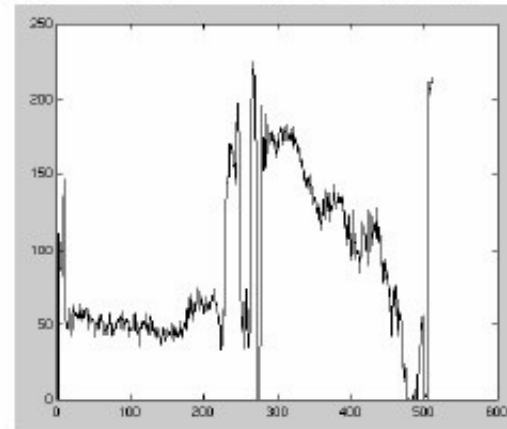
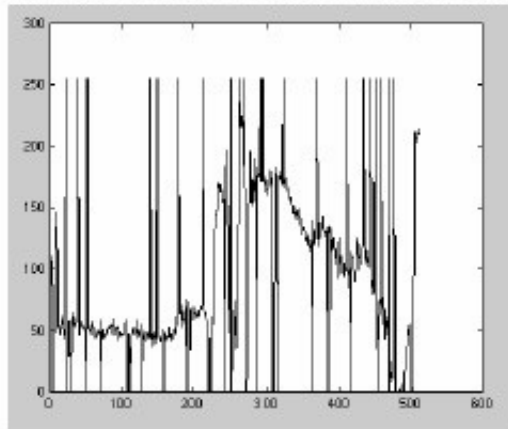
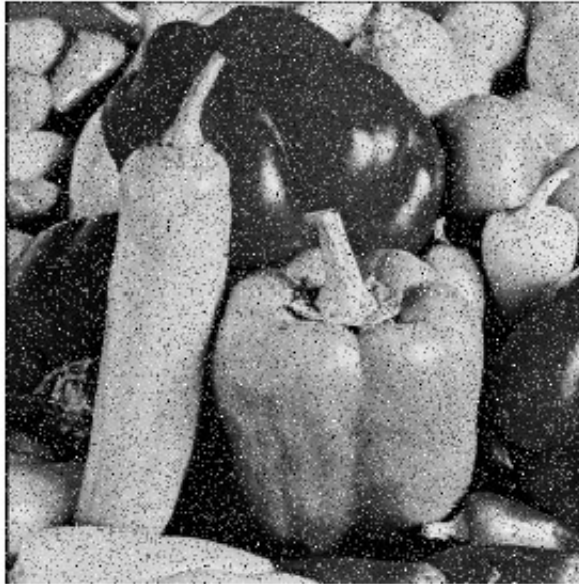
- Robustness to outliers

filters have width 5 :



Median filter

Salt-and-pepper noise Median filtered



`medfilt2(image, [h w])`

Median vs. Gaussian filtering

3x3

5x5

7x7

Gaussian



Median



JPEG Compression

Slides credit: Kayvon Fatahalian

JPEG Compression: The Big Ideas

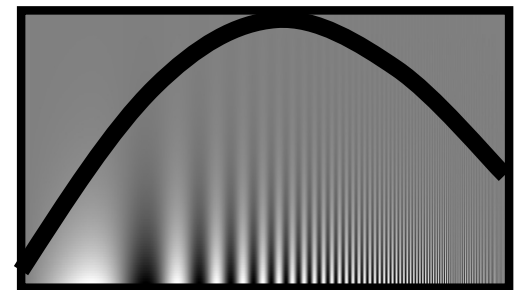
Low-frequency content is predominant in images of the real world

The human visual system is:

- Less sensitive to detail in chromaticity than in luminance
- Less sensitive to high frequency sources of error

Therefore, image compression of natural images can:

- Reduce perceived error by localizing error into high frequencies, and in chromaticity



Y'CbCr Color Space

Y'CbCr color space

- This is a perceptually-motivated color space akin to $L^*a^*b^*$ that we discussed in the color lecture
- Y' is luma (lightness), Cb and Cr are chroma channels (blue-yellow and red-green difference from gray)

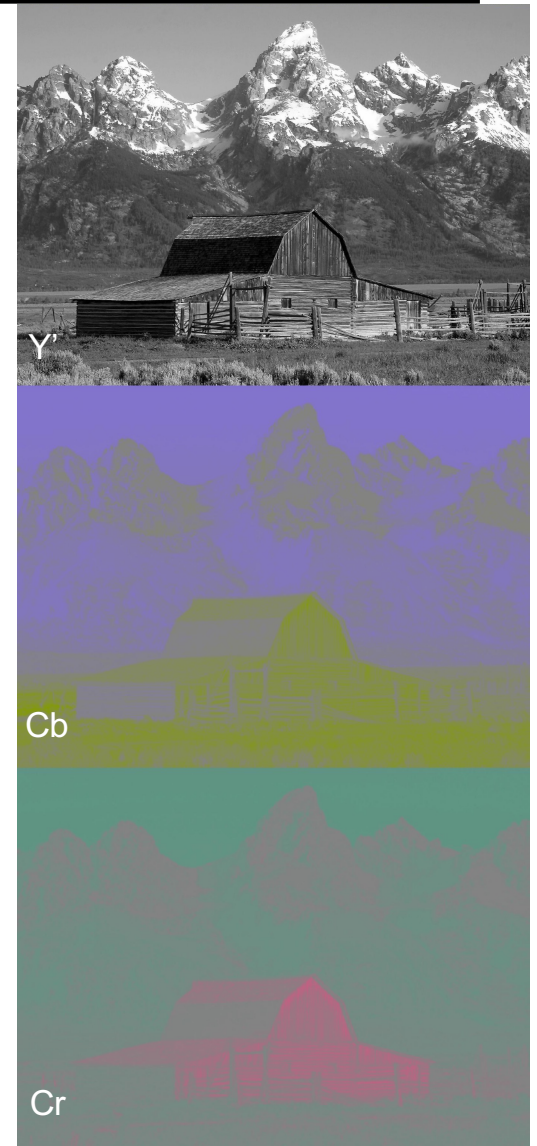


Image credit: Wikipedia

*Omitting discussion of nonlinear gamma encoding in Y' channel

Example Image



Original picture

Y' Only (Luma)



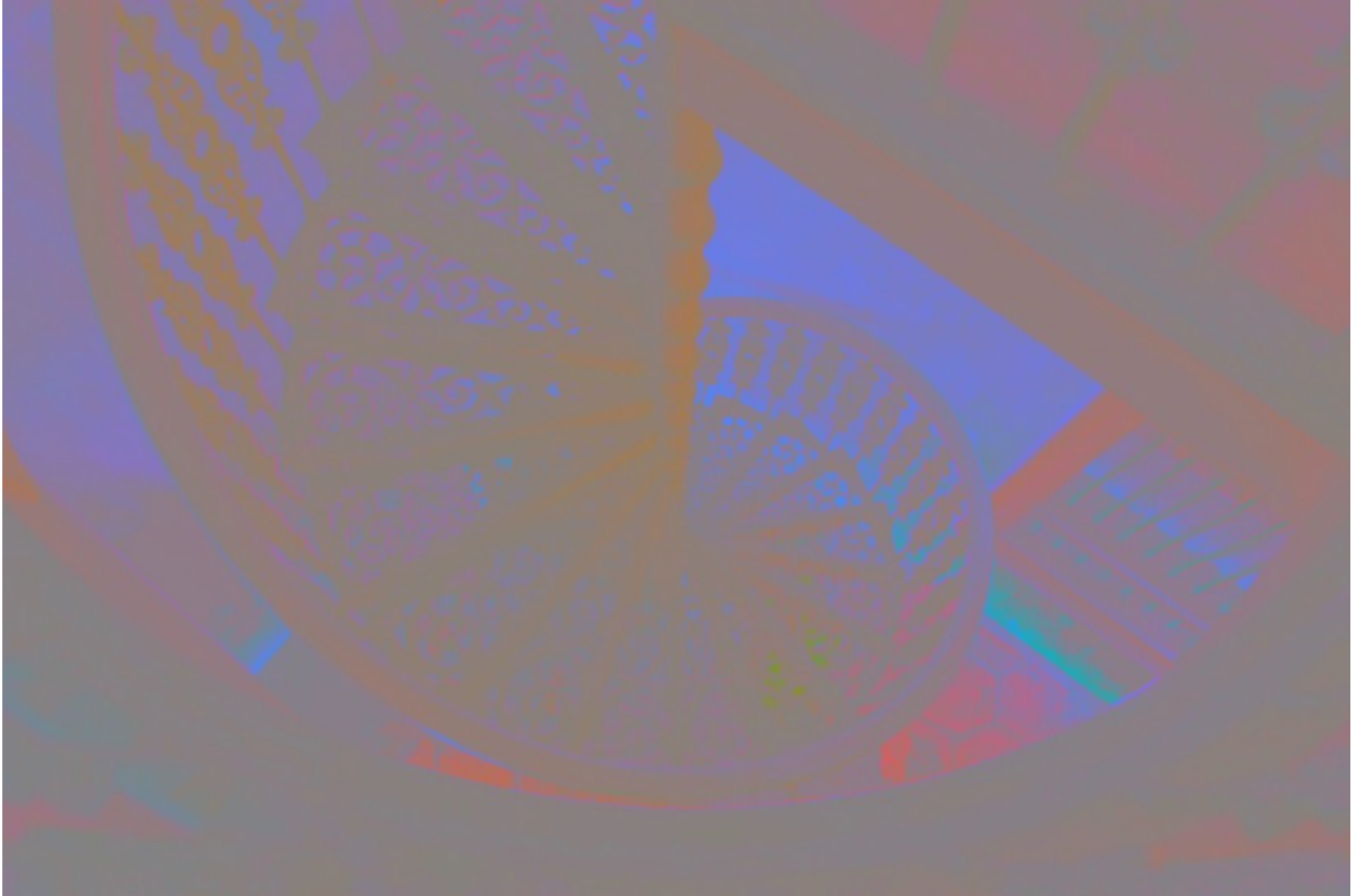
Luma channel

Downsampled Y'



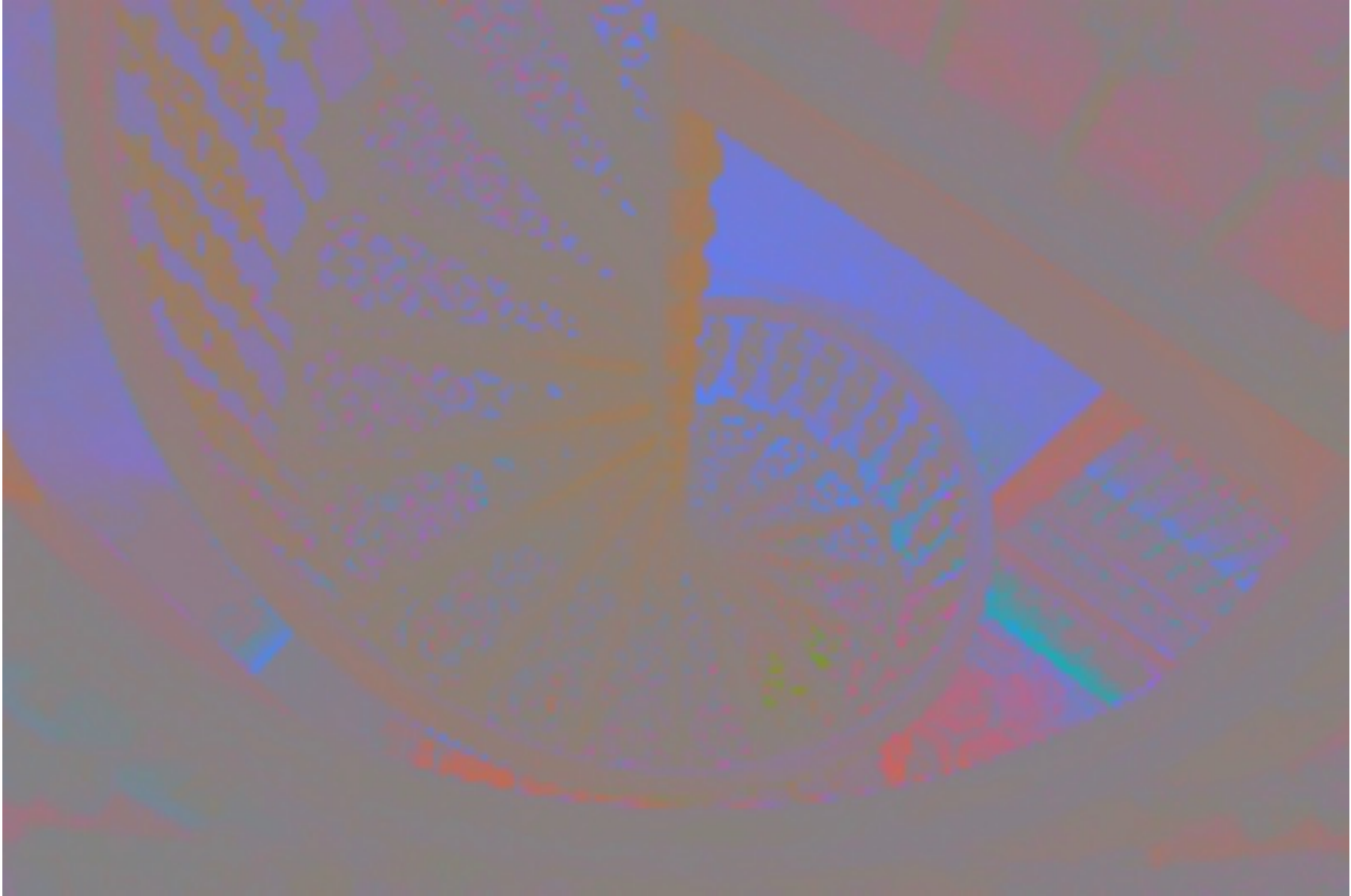
4x4 downsampled luma channel

CbCr Only (Chroma)



CbCr channels

Downsampled CbCr



4x4 downsampled CbCr channels

Example: Compression in Y' Channel



4x4 downsampled Y', full-resolution CbCr

Example: Compression in CbCr Channels



Full-resolution Y', 4x4 down sampled CbCr

Original Image

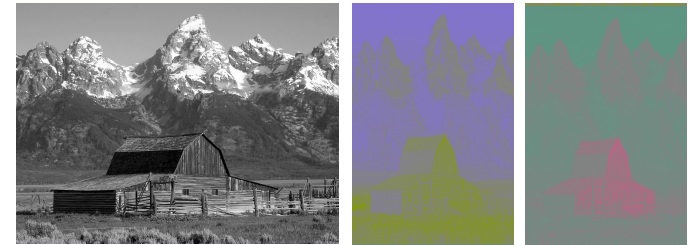


JPEG: Chroma Subsampling in Y'CbCr Space

Subsample chroma channels
(e.g. to 4:2:2 or 4:2:0 format)

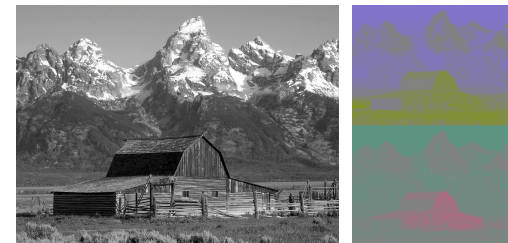
4:2:2 representation: (retain 2/3 values)

- Store Y' at full resolution
- Store Cb, Cr at half resolution in horizontal dimension



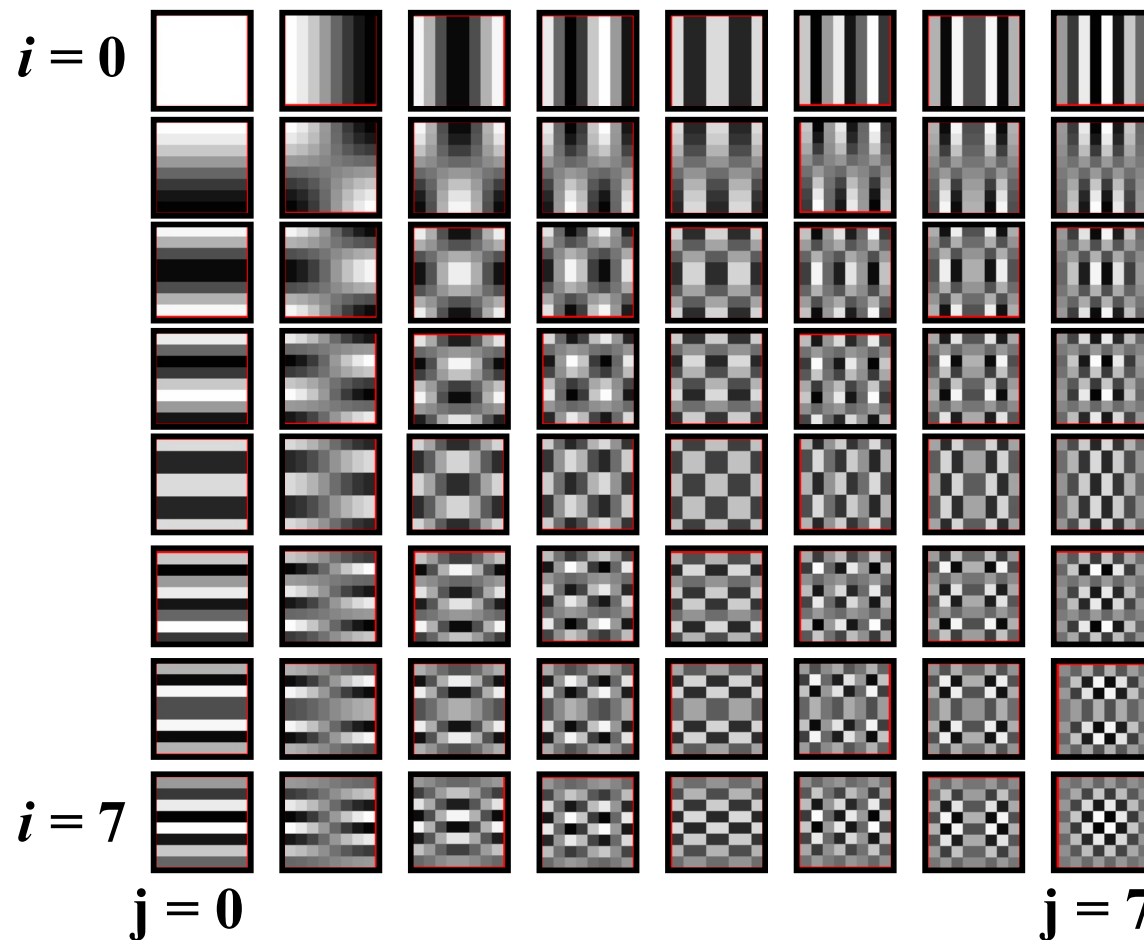
4:2:0 representation: (retain 1/2 values)

- Store Y' at full resolution
- Store Cb, Cr at half resolution in both dimensions



JPEG: Discrete Cosine Transform (DCT)

$$\text{basis}[i, j] = \cos \left[\pi \frac{i}{N} \left(x + \frac{1}{2} \right) \right] \times \cos \left[\pi \frac{j}{N} \left(y + \frac{1}{2} \right) \right]$$



In JPEG, Apply discrete cosine transform (DCT) to each 8x8 block of image values

DCT computes projection of image onto 64 basis functions:

$\text{basis}[i, j]$

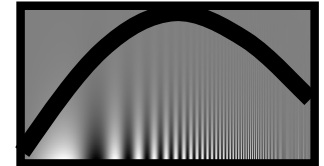
DCT applied to 8x8 pixel blocks of Y' channel, 16x16 pixel blocks of Cb, Cr (assuming 4:2:0)

JPEG Quantization: Save on High Frequencies

$$\begin{bmatrix} -415 & -30 & -61 & 27 & 56 & -20 & -2 & 0 \\ 4 & -22 & -61 & 10 & 13 & -7 & -9 & 5 \\ -47 & 7 & 77 & -25 & -29 & 10 & 5 & -6 \\ -49 & 12 & 34 & -15 & -10 & 6 & 2 & 2 \\ 12 & -7 & -13 & -4 & -2 & 2 & -3 & 3 \\ -8 & 3 & 2 & -6 & -2 & 1 & 4 & 2 \\ -1 & 0 & 0 & -2 & -1 & -3 & 4 & -1 \\ 0 & 0 & -1 & -4 & -1 & 0 & 1 & 2 \end{bmatrix} \div \begin{bmatrix} 16 & 11 & 10 & 16 & 24 & 40 & 51 & 61 \\ 12 & 12 & 14 & 19 & 26 & 58 & 60 & 55 \\ 14 & 13 & 16 & 24 & 40 & 57 & 69 & 56 \\ 14 & 17 & 22 & 29 & 51 & 87 & 80 & 62 \\ 18 & 22 & 37 & 56 & 68 & 109 & 103 & 77 \\ 24 & 35 & 55 & 64 & 81 & 104 & 113 & 92 \\ 49 & 64 & 78 & 87 & 103 & 121 & 120 & 101 \\ 72 & 92 & 95 & 98 & 112 & 100 & 103 & 99 \end{bmatrix}$$

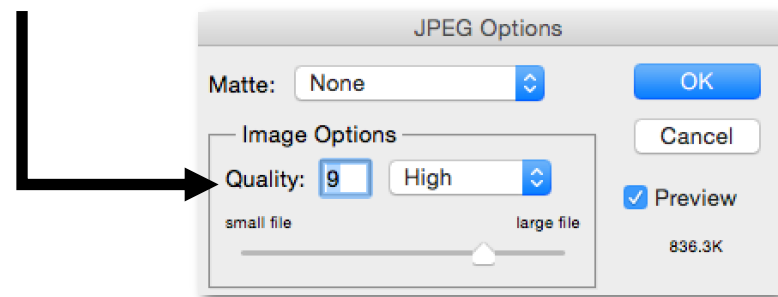
Result of DCT
(image encoded in cosine basis)

Quantization Matrix



$$= \begin{bmatrix} -26 & -3 & -6 & 2 & 2 & -1 & 0 & 0 \\ 0 & -2 & -4 & 1 & 1 & 0 & 0 & 0 \\ -3 & 1 & 5 & -1 & -1 & 0 & 0 & 0 \\ -4 & 1 & 2 & -1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Changing JPEG quality setting in your favorite photo app modifies this matrix ("lower quality" = higher values for elements in quantization matrix)



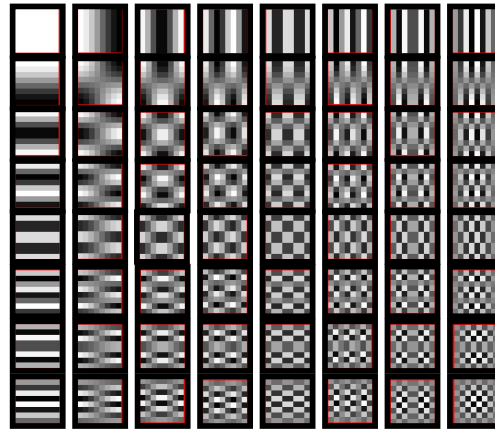
Quantization produces small values for coefficients (only a few bits needed per coefficient)

Observe: quantization zeros out many coefficients

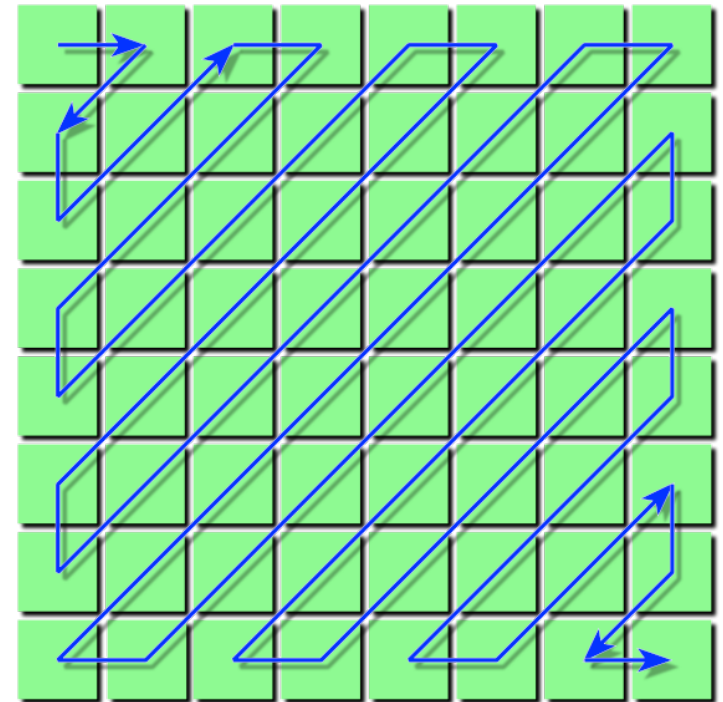
Lossless Compression of Quantized DCT Values

$$\begin{bmatrix} -26 & -3 & -6 & 2 & 2 & -1 & 0 & 0 \\ 0 & -2 & -4 & 1 & 1 & 0 & 0 & 0 \\ -3 & 1 & 5 & -1 & -1 & 0 & 0 & 0 \\ -4 & 1 & 2 & -1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Quantized DCT Values



Basis functions



Reordering

Entropy encoding: (lossless)

Reorder values

Run-length encode (RLE) 0's

Huffman encode non-zero values

JPEG Compression Summary

Convert image to Y'CbCr color space

Downsample CbCr (to 4:2:2 or 4:2:0) (information loss occurs here)

For each color channel (Y', Cb, Cr):

For each 8x8 block of values

- Compute DCT

- Quantize results (information loss occurs here)

- Reorder values

- Run-length encode 0-spans

- Huffman encode non-zero values

JPEG compression comparison



89k



12k

Theme: Exploit Perception in Visual Computing

JPEG is an example of a general theme of exploiting characteristics of human perception to build efficient visual computing systems

We are perceptually insensitive to color errors:

- Separate luminance from chrominance in color representations (e.g, Y'CrCb) and compress chrominance

We are less perceptually sensitive to high-frequency error

- Use a frequency-based encoding (cosine transform) and compress high-frequency values

We perceive lightness non-linearly (not discussed in this lecture)

- Encode pixel values non-linearly to match perceived brightness using gamma curve